

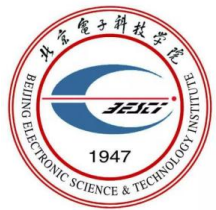
计算机网络

——2025-2026 第二学期

课程教师：周玉晨

联系方式：zhouyuchen@besti.edu.cn

时 间：2026年6月21日

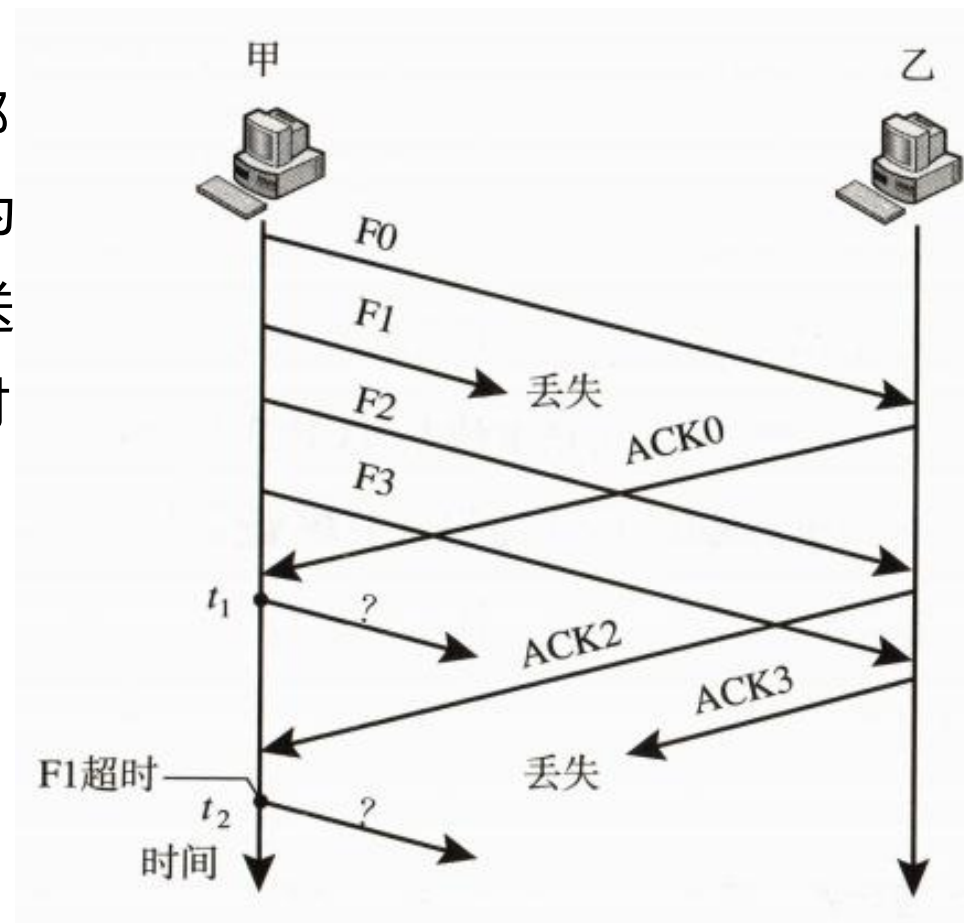


知识点复习

随堂练习

- 主机甲通过选择重传(SR)滑动窗口协议向主机乙发送帧的部分过程如图所示， F_x 为数据帧， ACK_x 为确认帧， x 是帧的序号。乙只对正确接收的数据帧进行独立确认，发送窗口与接收窗口大小相同且均为最大值。甲在 t_1 时刻和 t_2 时刻发送的数据帧分别是()。(在学习通上作答)

- A. F_1, F_3
- B. F_1, F_4
- C. F_3, F_1
- D. F_4, F_1





知识点复习

随堂练习

- 主机甲与主机乙之间建立了一个 TCP 连接,主机甲向主机乙发送了 3 个连续的 TCP 段,分别包含 100B、200B 和 400B 的有效载荷,第 3 个段的序号为 800。若主机乙仅正确收到第 1 个和第 3 个段,则主机乙发送给主机甲的确认序号是()。

(在学习通上作答)

- A. 100
- B. 500
- C. 600
- D. 1200



知识点复习

随堂练习

- 流量控制功能,实质上是由()执行的?
(在学习通上作答)
 - A. 发送方
 - B. 接收方
 - C. 发送方和接收方
 - D. 发送方和接收方间的中间节点



章节大纲

1	运输层协议概述
2	用户数据报协议UDP
3	传输控制协议TCP概述
4	可靠传输的工作原理
5	TCP报文段的首部格式
6	TCP可靠传输的实现
7	TCP 的流量控制
8	TCP的拥塞控制
9	TCP的运输连接管理



TCP的拥塞控制

拥塞避免

- **目的**：让拥塞窗口 cwnd 缓慢地增大，避免出现拥塞
- **拥塞窗口 cwnd 增大**：每经过一个往返时间 RTT（不管在此期间收到了多少确认），发送方的拥塞窗口 $cwnd = cwnd + 1$
- 具有**加法增大 AI** (Additive Increase) 特点：使拥塞窗口 cwnd 按**线性**规律缓慢增长

注意：

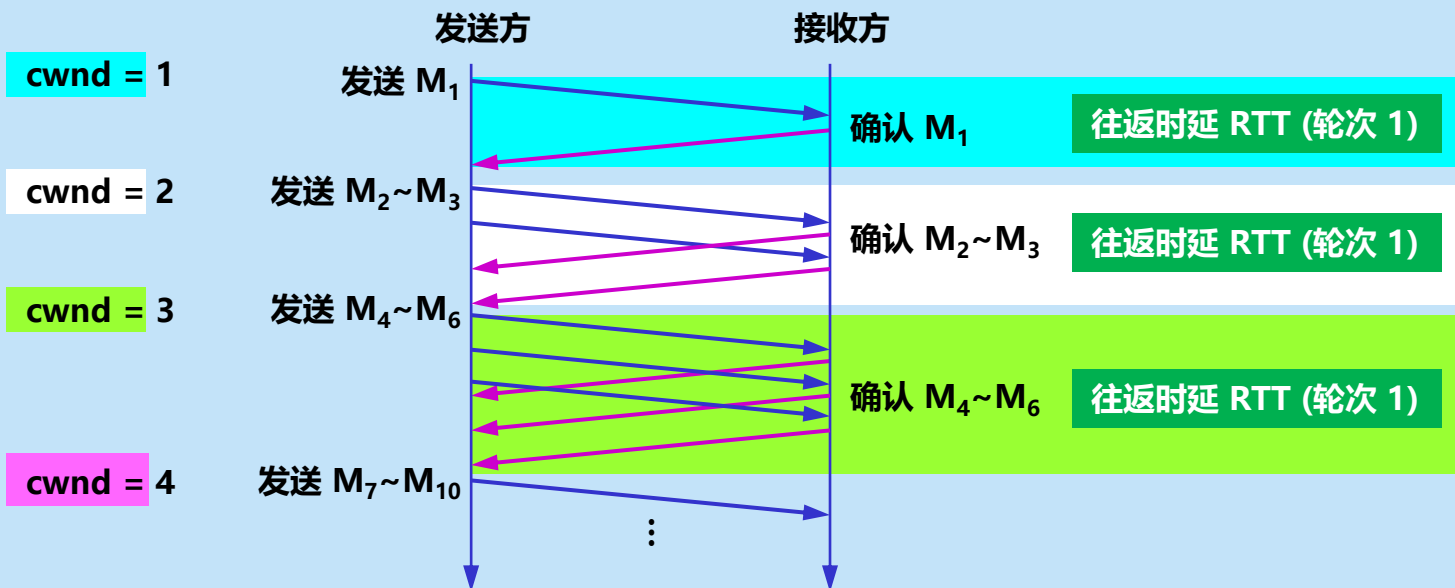
拥塞避免并非完全避免拥塞，而是让拥塞窗口增长得缓慢些，使网络不容易出现拥塞。



TCP的拥塞控制

拥塞避免

每经过一个往返时间 RTT，发送方就把拥塞窗口 $cwnd$ 加 1。





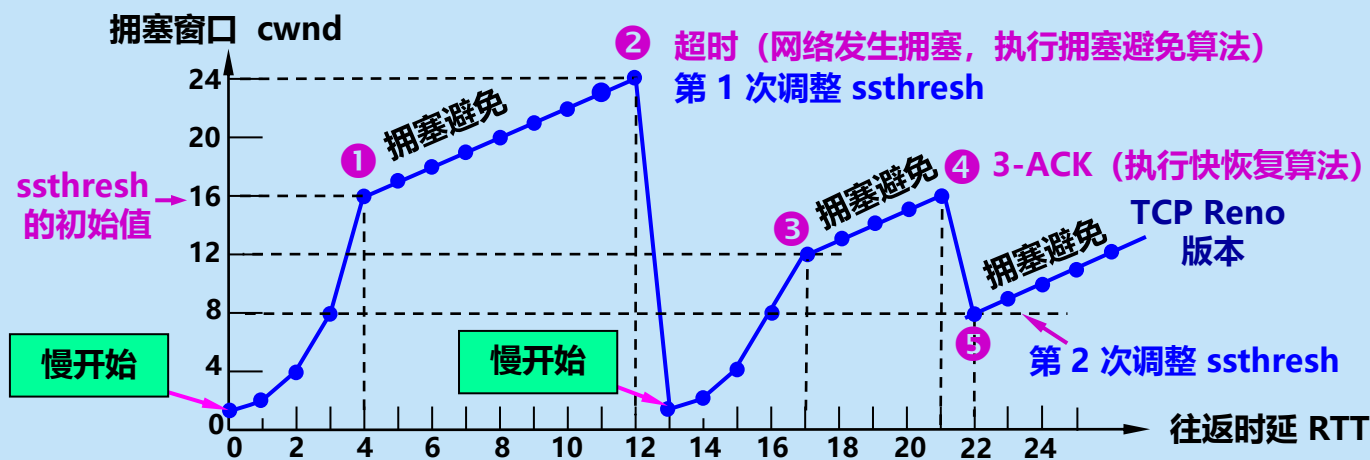
TCP的拥塞控制

当网络出现拥塞时

- 无论在慢开始阶段还是在拥塞避免阶段，只要发送方判断网络出现拥塞（重传定时器超时）：
 - $ssthresh = \max(cwnd/2, 2)$
 - $cwnd = 1$
 - 执行慢开始算法
- **目的：**迅速减少主机发送到网络中的分组数，使得发生拥塞的路由器有足够时间把队列中积压的分组处理完毕

TCP的拥塞控制

慢开始和拥塞避免算法的实现



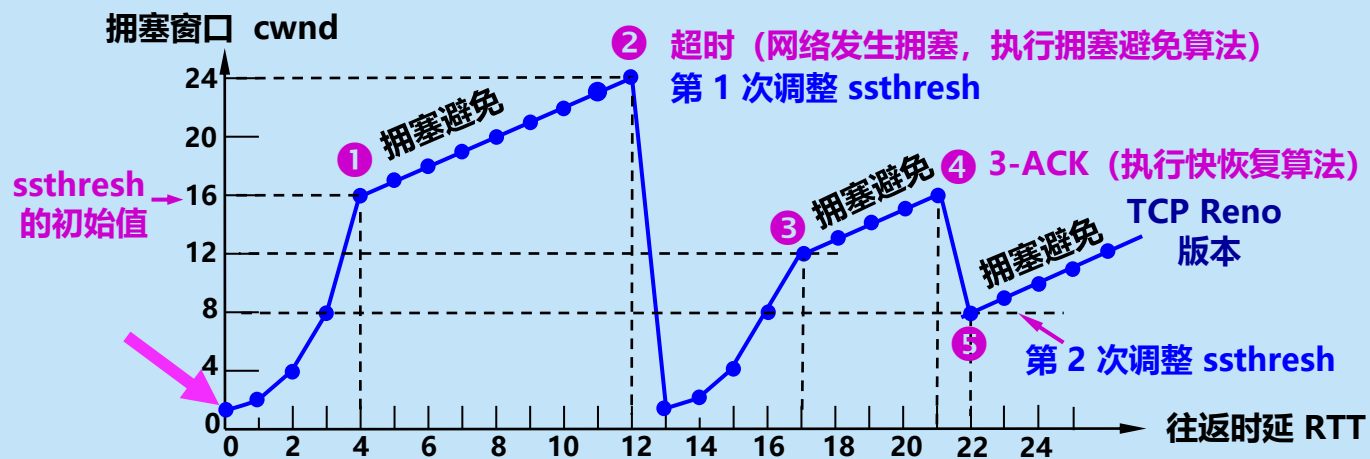
当 TCP 连接进行初始化时，将拥塞窗口置为 1（窗口单位不使用字节而使用报文段）。

将慢开始门限的初始值设置为 16 个报文段，即 $ssthresh = 16$ 。



TCP的拥塞控制

慢开始和拥塞避免算法的实现

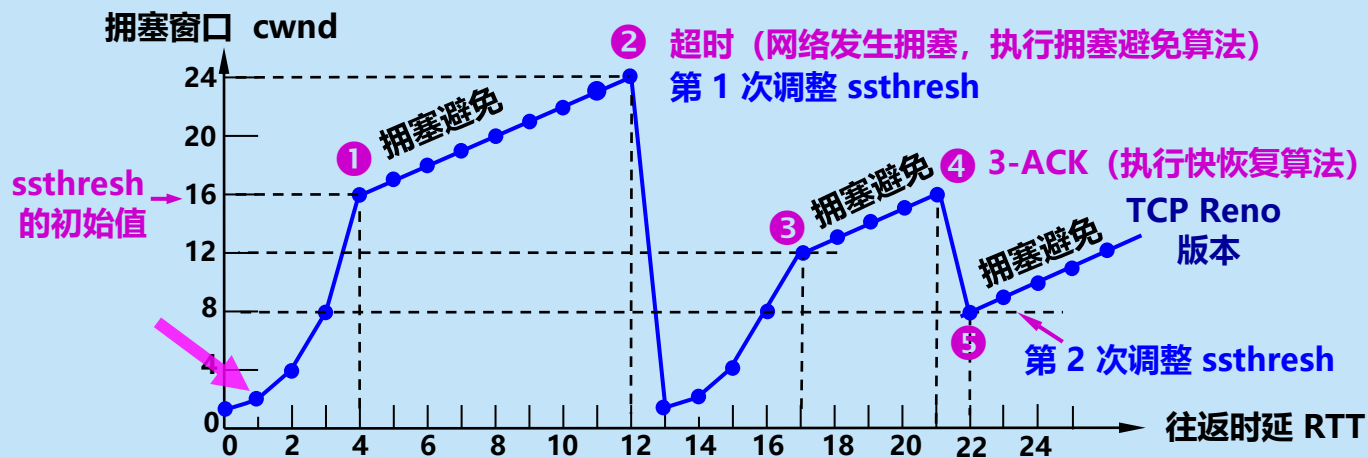


开始执行慢开始算法时, 拥塞窗口 $cwnd=1$, 发送第一个报文段。



TCP的拥塞控制

慢开始和拥塞避免算法的实现

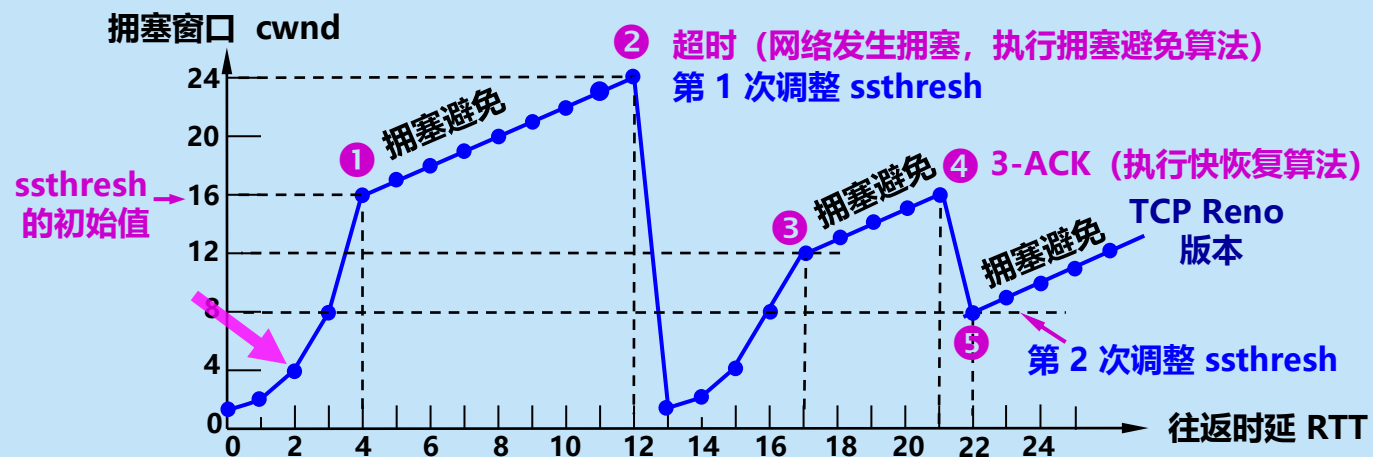


发送方每收到一个对新报文段的确认 ACK，就把拥塞窗口值加 1，因此拥塞窗口 cwnd 随着往返时延 RTT 按指数规律增长。



TCP的拥塞控制

慢开始和拥塞避免算法的实现

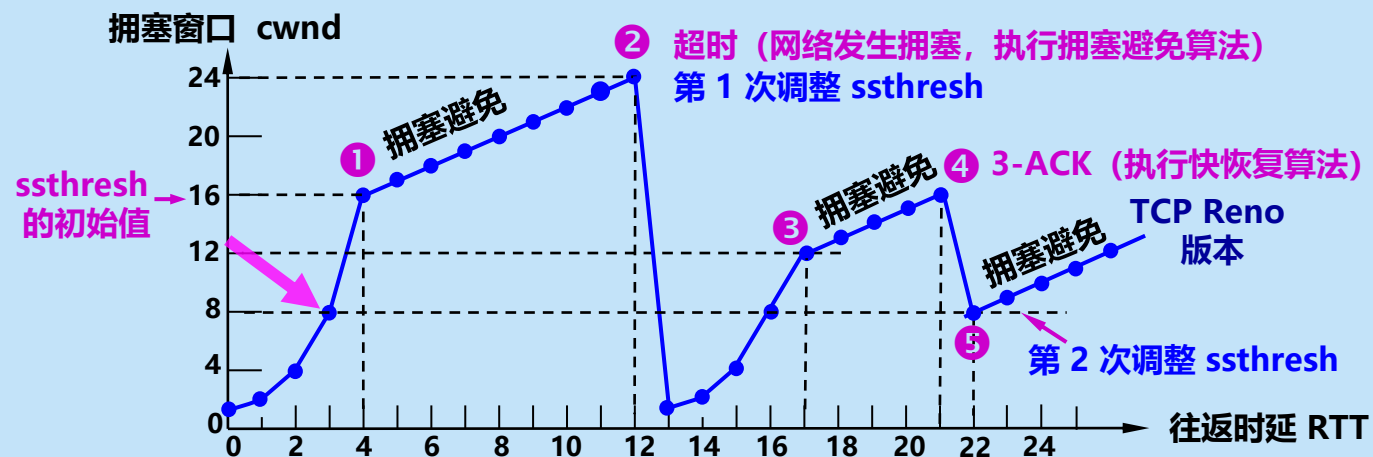


发送方每收到一个对新报文段的确认 ACK, 就把拥塞窗口值加 1, 因此拥塞窗口 cwnd 随着往返时延 RTT 按指数规律增长。



TCP的拥塞控制

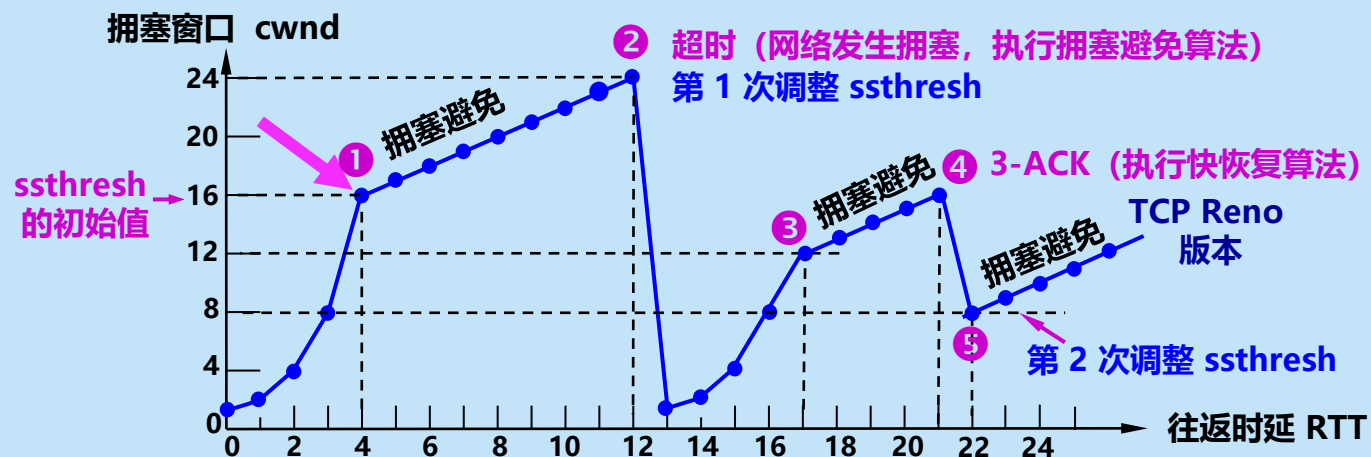
慢开始和拥塞避免算法的实现



发送方每收到一个对新报文段的确认 ACK，就把拥塞窗口值加 1，因此拥塞窗口 cwnd 随着往返时延 RTT 按指数规律增长。

TCP的拥塞控制

慢开始和拥塞避免算法的实现

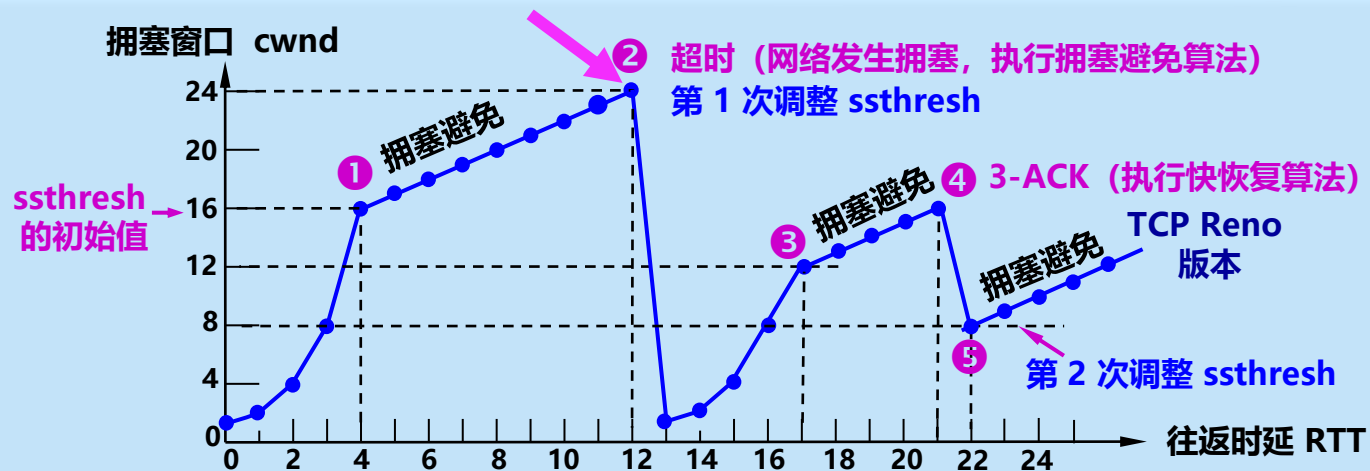


当拥塞窗口 cwnd 增长到慢开始门限值 ssthresh 时，改为执行拥塞避免算法，拥塞窗口按线性规律增长。



TCP的拥塞控制

慢开始和拥塞避免算法的实现

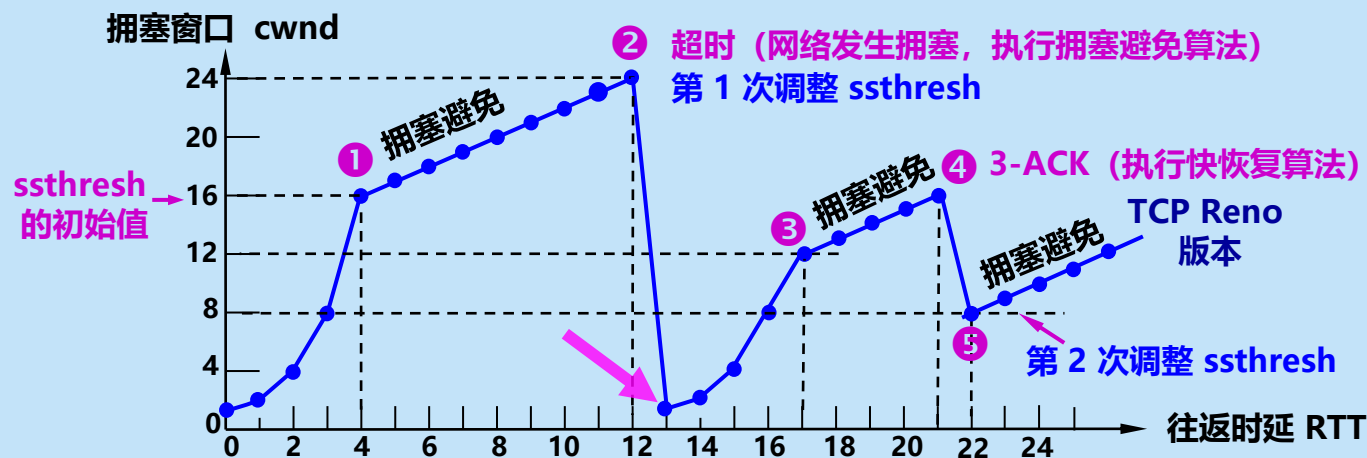


当拥塞窗口 $cwnd = 24$ 时, 网络出现了超时, 发送方判断为网络拥塞。调整门限值 $ssthresh = cwnd / 2 = 12$, 同时设置拥塞窗口 $cwnd = 1$, 进入慢开始阶段。



TCP的拥塞控制

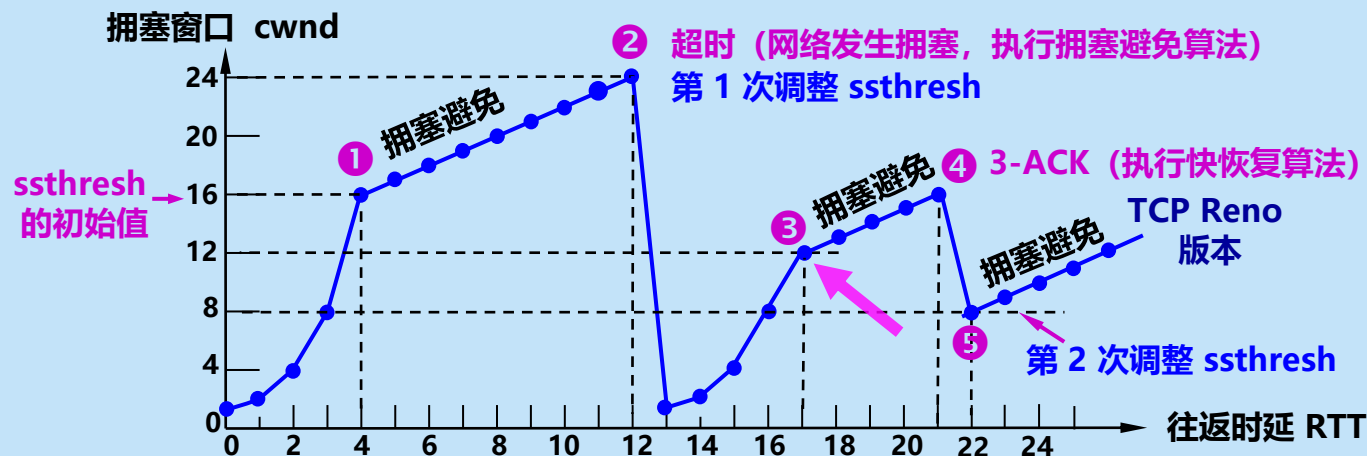
慢开始和拥塞避免算法的实现



当拥塞窗口 $cwnd = 24$ 时，网络出现了超时，发送方判断为网络拥塞。调整门限值 $ssthresh = cwnd / 2 = 12$ ，同时设置拥塞窗口 $cwnd = 1$ ，进入慢开始阶段。

TCP的拥塞控制

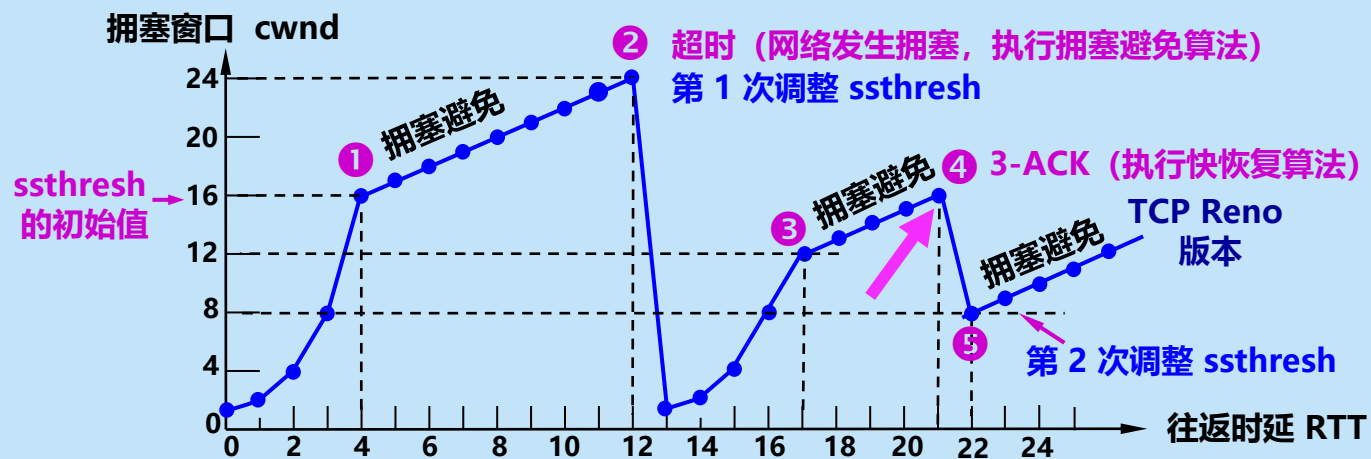
慢开始和拥塞避免算法的实现



按照慢开始算法, 发送方每收到一个对新报文段的确认 ACK, 就把拥塞窗口值加 1。当拥塞窗口 $cwnd = ssthresh = 12$ 时, 改为执行拥塞避免算法, 拥塞窗口按线性规律增大。

TCP的拥塞控制

慢开始和拥塞避免算法的实现



当拥塞窗口 $cwnd = 16$ 时, 发送方连续收到 3 个对同一个报文段的重复确认 (记为 3-ACK)。发送方改为执行**快重传**和**快恢复**算法。



TCP的拥塞控制

快重传 FR (Fast Retransmission) 算法

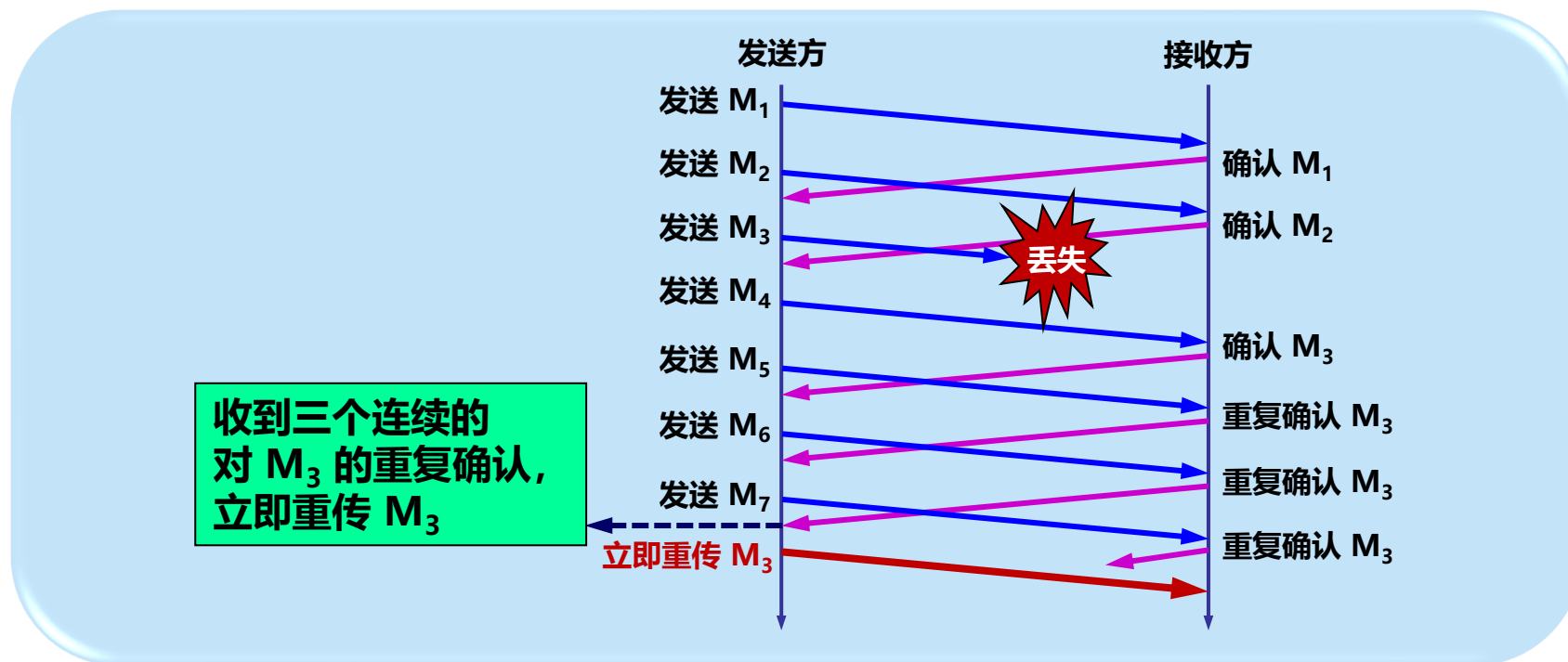
- **目的：**让发送方**尽早**知道发生了个别报文段的丢失
- 发送方只要连续收到**三个重复的确认**，就**立即进行重传**（即“快重传”），这样就不会出现超时
- 使用快重传可以使整个网络的吞吐量提高约 20%
- 快重传算法要求接收方**立即发送确认**，即使收到了失序的报文段，也要立即发出对已收到的报文段的重复确认

注意：快重传并非取消重传计时器，而是在某些情况下可以更早地（**更快地**）重传丢失的报文段。



TCP的拥塞控制

快重传 FR (Fast Retransmission) 算法





TCP的拥塞控制

快恢复 FR (Fast Recovery)算法

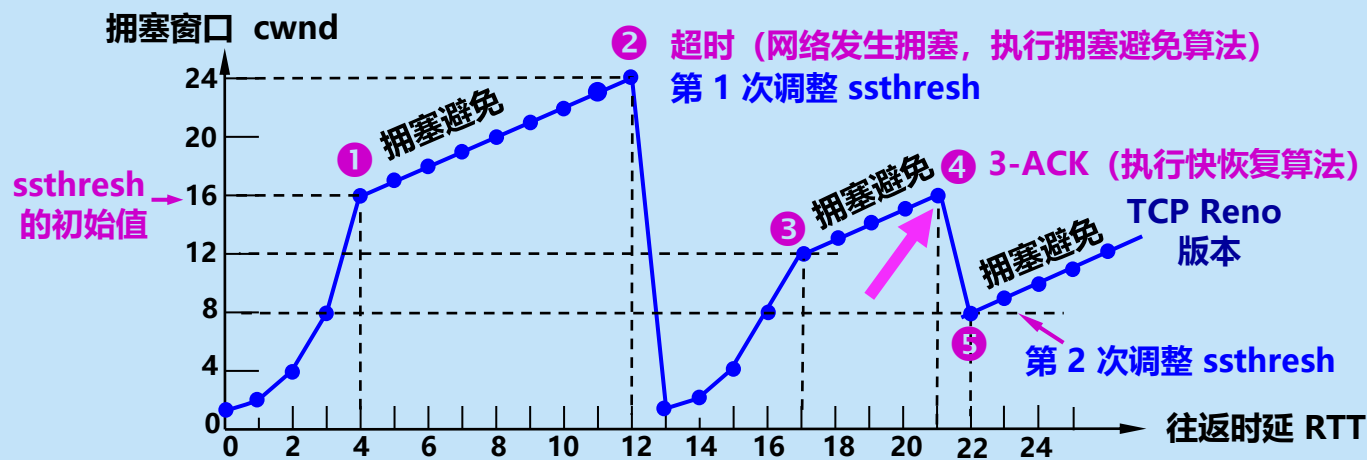
- 当发送端收到连续三个重复的确认时，**不执行**慢开始算法，而是**执行快恢复算法 FR** (Fast Recovery) 算法
 - 慢开始门限 $ssthresh = \text{当前拥塞窗口 } cwnd / 2$
 - **乘法减小 MD** (Multiplicative Decrease) 拥塞窗口
 - 新拥塞窗口 $cwnd = \text{慢开始门限 } ssthresh$
 - 执行拥塞避免算法，使拥塞窗口缓慢地**线性增大 (加法增大 AI)**

二者合在一起就是所谓的 **AIMD** 算法，使 TCP 性能有明显改进。



TCP的拥塞控制

慢开始和拥塞避免算法

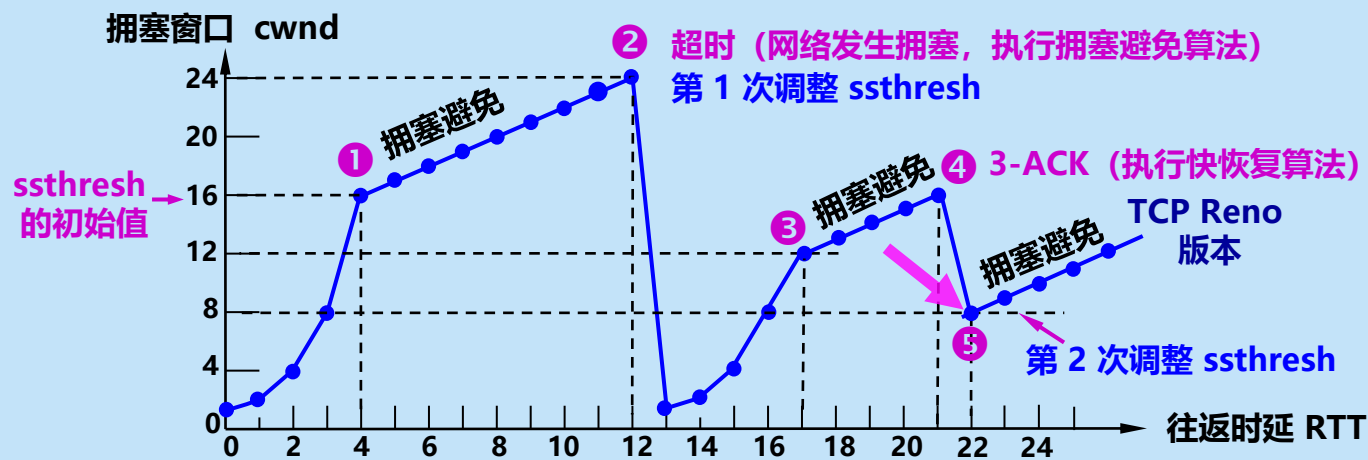


当拥塞窗口 $cwnd = 16$ 时，发送方连续收到 3 个对同一个报文段的重复确认（记为 3-ACK）。发送方改为执行快重传和快恢复算法。



TCP的拥塞控制

慢开始和拥塞避免算法



执行快重传和快恢复算法: 发送方调整门限值 $ssthresh = cwnd / 2 = 8$, 设置拥塞窗口 $cwnd = ssthresh = 8$, 开始执行拥塞避免算法。



TCP的拥塞控制

练习

- 如果算法的思路是由小到大逐渐指数级增大拥塞窗口数值，这种拥塞控制算法是？
 - A. 慢开始
 - B. 拥塞避免
 - C. 快重传
 - D. 快恢复



TCP的拥塞控制

练习

- 慢开始门限 $ssthresh$ 的作用是?
 - A. 当 $cwnd < ssthresh$ 时, 使用慢开始算法
 - B. 当 $cwnd > ssthresh$ 时, 停止使用慢开始算法而改用拥塞避免算法
 - C. 当 $cwnd > ssthresh$ 时, 使用慢开始算法
 - D. 当 $cwnd < ssthresh$ 时, 停止使用慢开始算法而改用拥塞避免算法



TCP的拥塞控制

练习

- 让拥塞窗口 cwnd 缓慢地增大, 即每经过一个往返时间 RTT 就把发送方的拥塞窗口 cwnd 加 1, 使拥塞窗口 cwnd 线性缓慢增长, 这种拥塞算法是?
 - A. 慢开始
 - B. 拥塞避免
 - C. 快重传
 - D. 快恢复



TCP的拥塞控制

练习

- 当发送方连续收到三个重复确认，就知道接收方没有收到某一报文段，因而立即进行重传，这样发送方也不就会误认为出现了网络拥塞。这种拥塞控制算法是？
 - A. 慢开始
 - B. 拥塞避免
 - C. 快重传
 - D. 快恢复



TCP的拥塞控制

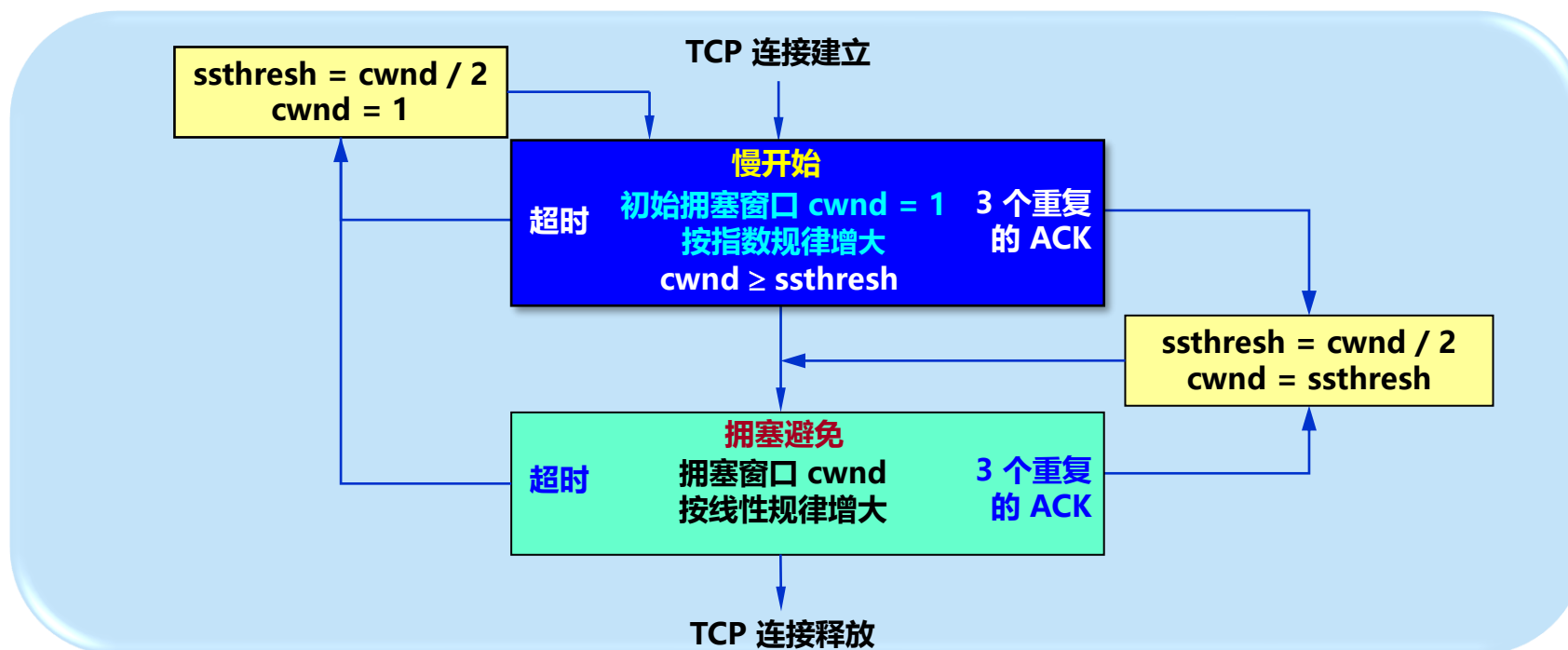
练习

- 当发送端收到连续三个重复的确认时，由于发送方现在认为网络很可能没有发生拥塞，因此现在不执行慢开始算法，而是执行_____算法？
 - A. 慢开始
 - B. 拥塞避免
 - C. 快重传
 - D. 快恢复



TCP的拥塞控制

TCP拥塞控制流程图





TCP的拥塞控制

发送窗口的上限值（若同时采用流量控制和拥塞控制）

- 当 $rwnd < cwnd$ 时，是接收方的接收能力限制发送窗口的最大值
- 当 $cwnd < rwnd$ 时，是网络拥塞限制发送窗口的最大值

$$\text{发送窗口的上限值} = \text{Min} [rwnd, cwnd] \quad (5-9)$$



TCP的拥塞控制

讨论

- 流量控制和拥塞控制的最大区别是什么？
- 发送窗口的大小，取决于流量控制，还是拥塞控制？



TCP的拥塞控制

问题

- 设TCP的sssthresh 的初始值为8(单位为报文段)。当拥塞窗口上升到 12 时网络发生了超时, TCP 使用慢开始和拥塞避免。试分别求出第 1 轮次到第 15 轮次传输的各拥塞窗口大小。你能说明拥塞窗口每一次变化的原因吗?

- 答: 第一次cwnd = 1

第二次cwnd = 2

第三次cwnd = 4

第四次cwnd = 8, 慢开始改成拥塞避免

第五次cwnd = 9

第六次cwnd = 10

第七次cwnd = 11

第八次cwnd = 12, 超时 sssthresh = 6

第九次cwnd = 1

第十次cwnd = 2

第十一次cwnd = 4

第十二次cwnd = 6, 慢开始改成拥塞避免

第十三次cwnd = 7

第十四次cwnd = 8 第十五次cwnd = 9



TCP的拥塞控制

问题

- TCP的拥塞窗口 cwnd 大小与传输轮次n的关系如下所示

cwnd	1	2	4	8	16	32	33	34	35	36	37	38	39
n	1	2	3	4	5	6	7	8	9	10	11	12	13
cwnd	40	41	42	21	22	23	24	25	26	1	2	4	8
n	14	15	16	17	18	19	20	21	22	23	24	25	26

- (1) 试画出拥塞窗口与传输次的关系曲线。
- (2) 指明TCP工作在慢开始阶段的时间间隔
- (3) TCP工作在拥塞避免阶段的时间间隔
- (4) 在第 16轮次和第 22次之后发送是通过收到三个重复的确认还是通过超时检测到丢失了报文段
- (5) 在第1轮次第18 轮次和第24 轮次发送时，门限 ssthresh 分别被设置为多大
- (6) 在第几轮发送第70个报文？
- (7) 假定在第 26 轮次之后收到了三个重复的确认，因而检测出了报文段的丢失，那么拥塞窗口 cwnd 和门限ssthresh 应设置为多大？



TCP的拥塞控制

问题

慢开始

拥塞避免

cwnd	1	2	4	8	16	32	33	34	35	36	37	38	39
n	1	2	3	4	5	6	7	8	9	10	11	12	13
cwnd	40	41	42	21	22	23	24	25	26	1	2	4	8
n	14	15	16	17	18	19	20	21	22	23	24	25	26

- (1) 试画出拥塞窗口与传输次的关系曲线。
- (2) 指明TCP工作在慢开始阶段的时间间隔
- (3) TCP工作在拥塞避免阶段的时间间隔
- (4) 在第 16 轮次和第 22 次之后发送是通过收到三个重复的确认还是通过超时检测到丢失了报文段
- (5) 在第 1 轮次、第 18 轮次和第 24 轮次发送时，门限 $ssthresh$ 分别被设置为多大
- (6) 在第几轮发送第 70 个报文？
- (7) 假定在第 26 轮次之后收到了三个重复的确认，因而检测出了报文段的丢失，那么拥塞窗口 $cwnd$ 和门限 $ssthresh$ 应设置为多大？



TCP的拥塞控制

问题

- 若主机甲与主机乙已建立一条 TCP 连接，最大段长 (MSS) 为 1KB，往返时间 (RTT) 为 2ms，则在不出现拥塞的前提下，拥塞窗口从 8kB 增长到 32K 所需的**最长时间**是
- A. 4ms
- B. 8ms
- C. 24ms
- D. 48ms



TCP的拥塞控制

问题

- 用TCP通过速率为1Gbit/s的链路传送一个10MB的文件。假定链路的往返时间 $RTT=50ms$ 。TCP选用了窗口扩大选项，使窗口达到可选用的最大值。在接收端TCP的接收窗口为1MB（保持不变），而发送端采用拥塞控制算法，从慢开始传送。假定拥塞窗口以分组为单位计算，在一开始发送1个分组，而每个分组长度都是1KB。假定网络不会发生拥塞和分组丢失，并且发送端发送数据的速率足够快，因此发送时延可以忽略不计，而接收端每一次收完一批分组后就立即发送确认ACK分组。
 - (1) 经过多少个RTT后，发送窗口大小达到1MB？
 - (2) 发送端把整个10MB文件传送成功共需要经过多少个RTT？传送成功是指发送完整个文件，并收到所有的确认。TCP扩大的窗口够用吗？
 - (3) 根据整个文件发送成功所花费的时间（包括收到所有的确认），计算此传送链路的有效吞吐率。链路带宽的利用率使多少？



TCP的拥塞控制

问题

- 用TCP通过速率为1Gbit/s的链路传送一个10MB的文件。假定链路的往返时间RTT=50ms。TCP选用了窗口扩大选项，使窗口达到可选用的最大值。在接收端TCP的接收窗口为1MB（保持不变），而发送端采用拥塞控制算法，从慢开始传送。假定拥塞窗口以分组为单位计算，在一开始发送1个分组，而每个分组长度都是1KB。假定网络不会发生拥塞和分组丢失，并且发送端发送数据的速率足够快，因此发送时延可以忽略不计，而接收端每一次收完一批分组后就立即发送确认ACK分组。

(1) 经过多少个RTT后，发送窗口大小达到1MB？

解：刚开始发送窗口：1KB

1个RTT后，发送窗口：2KB

2个RTT后，发送窗口：4KB

.....

N个RTT后，发送窗口： 2^N KB

所以，第10个RTT后，发送窗口： 2^{10} KB=1MB



TCP的拥塞控制

问题

- 用TCP通过速率为1Gbit/s的链路传送一个10MB的文件。假定链路的往返时间 $RTT=50ms$ 。TCP选用了窗口扩大选项，使窗口达到可选用的最大值。在接收端TCP的接收窗口为1MB（保持不变），而发送端采用拥塞控制算法，从慢开始传送。假定拥塞窗口以分组为单位计算，在一开始发送1个分组，而每个分组长度都是1KB。假定网络不会发生拥塞和分组丢失，并且发送端发送数据的速率足够快，因此发送时延可以忽略不计，而接收端每一次收完一批分组后就立即发送确认ACK分组。

(2) 发送端把整个10MB文件传送成功共需要经过多少个RTT？传送成功是指发送完整个文件，并收到所有的确认。TCP扩大的窗口够用吗？

解：第一个RTT发送了1KB

第二个RTT发送了2KB，共发送了3KB

第三个RTT发送了4KB，共发送了7KB

.....

第N个RTT发送了 $2^{N-1}KB$ ，共发送了 $(2^N-1) KB$

所以，第13个RTT发送了 $2^{12}KB=4MB$ ，共发送了 $(2^{13}-1) KB= 8MB - 1KB$

第14个RTT发送了 $2MB+1KB$ ，共发送了10MB

第14个RRT开始时窗口长度为 $2^{13}KB=8MB$ ，所以扩大窗口够用 **38**



TCP的拥塞控制

问题

- 用TCP通过速率为1Gbit/s的链路传送一个10MB的文件。假定链路的往返时间RTT=50ms。TCP选用了窗口扩大选项，使窗口达到可选用的最大值。在接收端TCP的接收窗口为1MB（保持不变），而发送端采用拥塞控制算法，从慢开始传送。假定拥塞窗口以分组为单位计算，在一开始发送1个分组，而每个分组长度都是1KB。假定网络不会发生拥塞和分组丢失，并且发送端发送数据的速率足够快，因此发送时延可以忽略不计，而接收端每一次收完一批分组后就立即发送确认ACK分组。

(3) 根据整个文件发送成功所花费的时间（包括收到所有的确认），计算此传送链路的有效吞吐率。链路带宽的利用率使多少？

解：吞吐率： $10\text{MB} / 14\text{RTT} = (10 \times 2^{20} \times 8) \text{ bit} / (14 \times 0.05) \text{ s} = 119.8\text{Mbit/s}$

带宽利用率： $119.8\text{Mbit/s} / 1\text{Gbit/s} = 11.98\%$



TCP的拥塞控制

主动队列管理AQM

- TCP 拥塞控制和网络层采取的策略有密切联系
- 例如：
 - 若路由器对某些分组的处理时间特别长，就可能引起发送方 TCP 超时，对这些报文段进行重传
 - 重传会使 TCP 连接的发送端认为在网络中发生了拥塞，但实际上网络并没有发生拥塞
- 对 TCP 拥塞控制影响最大的就是路由器的分组丢弃策略



TCP的拥塞控制

“先进先出” FIFO 处理规则

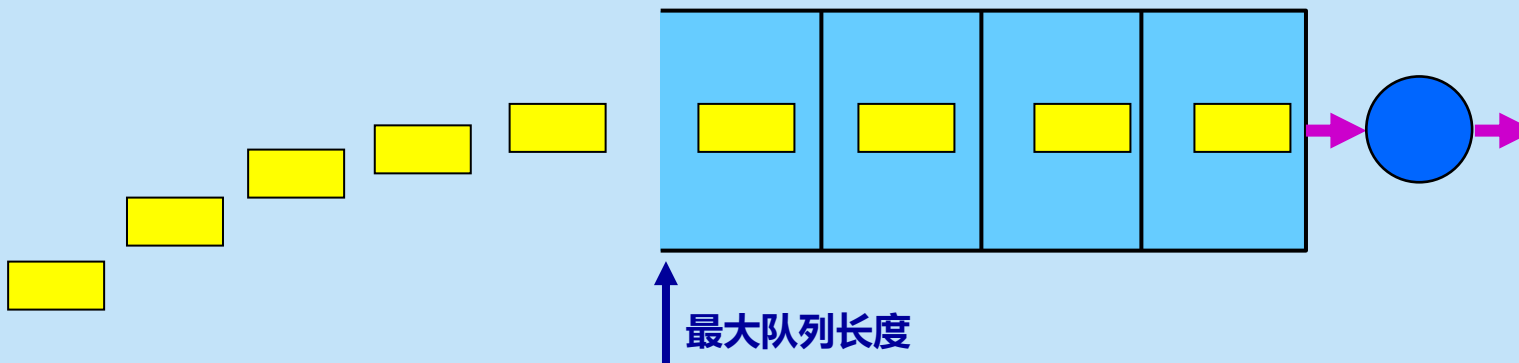
- **先进先出FIFO** (First In First Out) 处理规则
- **尾部丢弃策略** (tail-drop policy): 当队列已满时, 以后到达的所有分组 (如果能够继续排队, 这些分组都将排在队列的尾部) 将都被丢弃
- 路由器的尾部丢弃往往会导致一连串分组的丢失, 这就使发送方出现超时重传, 使 TCP 进入拥塞控制的慢开始状态, 结果使 TCP 连接的发送方突然把数据的发送速率降低到很小的数值



TCP的拥塞控制

先进先出 (FIFO) 处理规则与尾部丢弃策略

在最简单的情况下，路由器队列通常采用
先进先出 (FIFO) 处理规则与**尾部丢弃策略 (tail-drop policy)**。
当队列已满时，以后到达的所有分组将都被丢弃。

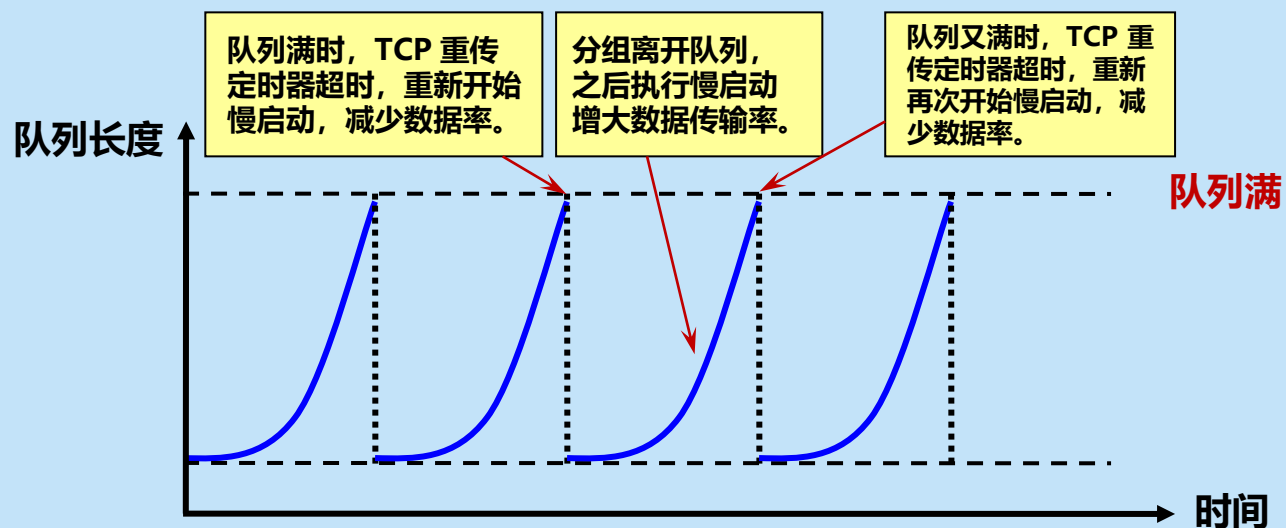




TCP的拥塞控制

先进先出（FIFO）处理规则与尾部丢弃策略

分组丢弃使发送方出现超时重传，使 TCP 连接进入慢开始状态。

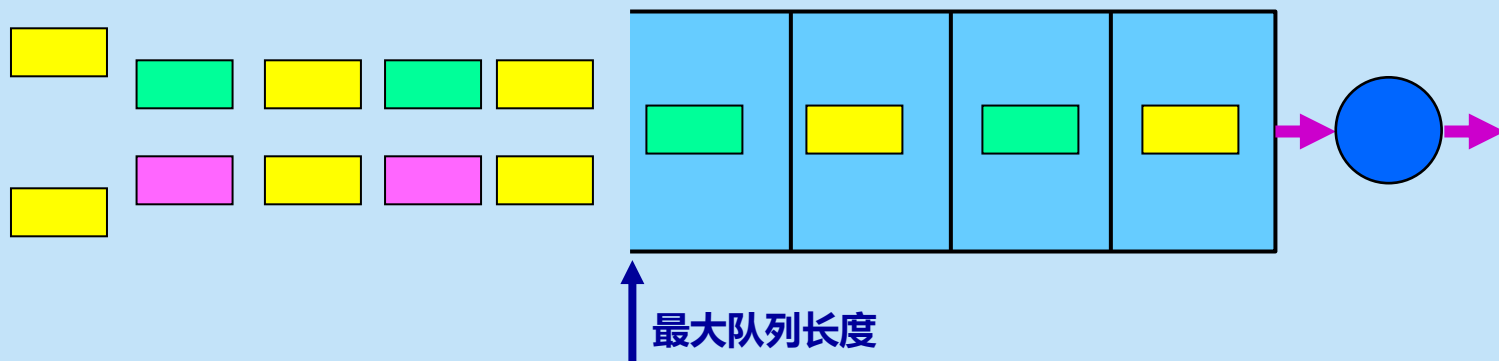




TCP的拥塞控制

严重问题：全局同步

若路由器进行了尾部丢弃，
所有到达的分组都被丢弃，不论它们属于哪个 TCP 连接。

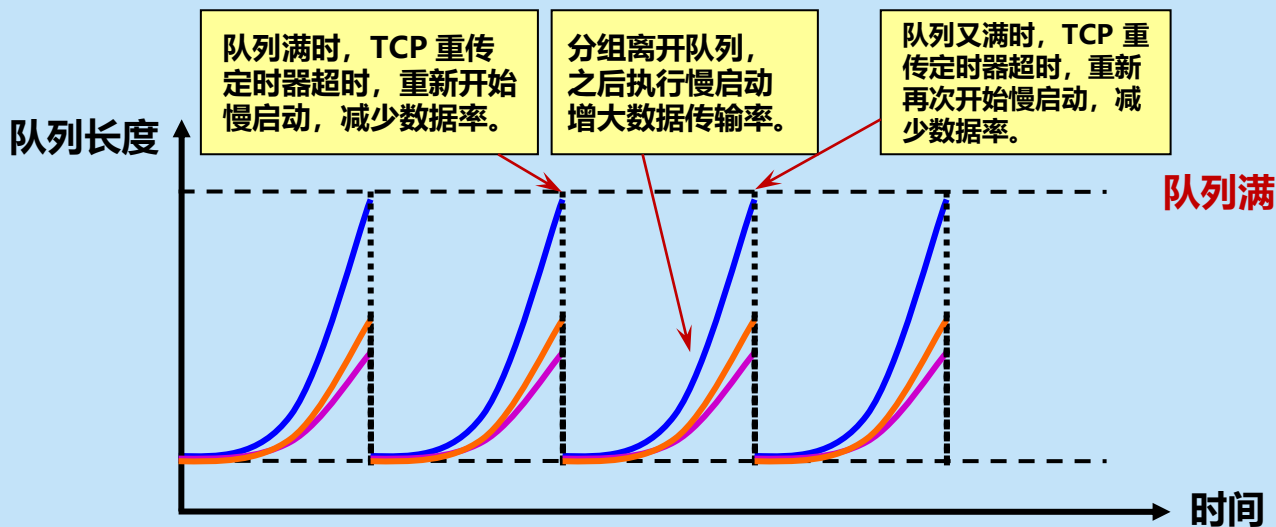




TCP的拥塞控制

严重问题：全局同步

分组丢弃使发送方出现超时重传，使多个 TCP 连接**同时**进入慢开始状态，发生**全局同步** (global synchronization)。





TCP的拥塞控制

主动队列管理AQM

- 1998 年提出了主动队列管理 AQM (Active Queue Management)
- **主动：** 不要等到路由器的队列长度已经达到最大值时才不得不丢弃后面到达的分组，而是在队列长度**达到某个值得警惕的数值时**（即当网络拥塞有了某些拥塞征兆时），就**主动丢弃**到达的分组
- AQM 可以有不同实现方法，其中曾流行多年的就是**随机早期检测 RED** (Random Early Detection)



TCP的拥塞控制

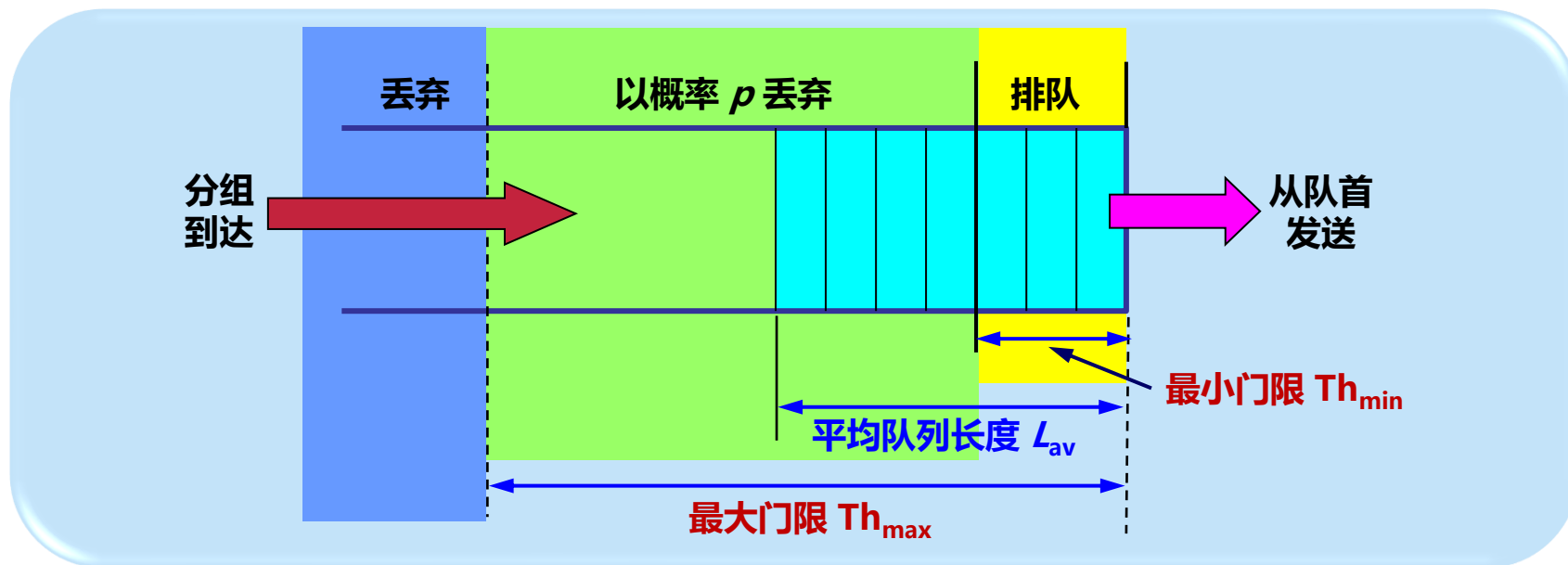
随机早期检测 RED

- 路由器队列维持两个参数：
 - 队列长度**最小**门限 TH_{min}
 - 队列长度**最大**门限 TH_{max}
- RED 对每一个到达的分组都先计算平均队列长度 L_{AV}
 - 若平均队列长度**小于最小**门限 TH_{min} ，则将新到达的分组放入队列进行排队
 - 若平均队列长度**超过最大**门限 TH_{max} ，则将新到达的分组丢弃
 - 若平均队列长度**介于**在最小门限 TH_{min} 和最大门限 TH_{ax} **之间**，则按照某一**概率 p** 将新到达的分组丢弃

TCP的拥塞控制

随机早期检测 RED

RED 路由器到达队列维持**两个参数**: Th_{min} , Th_{max} , 分成为**三个区域**:



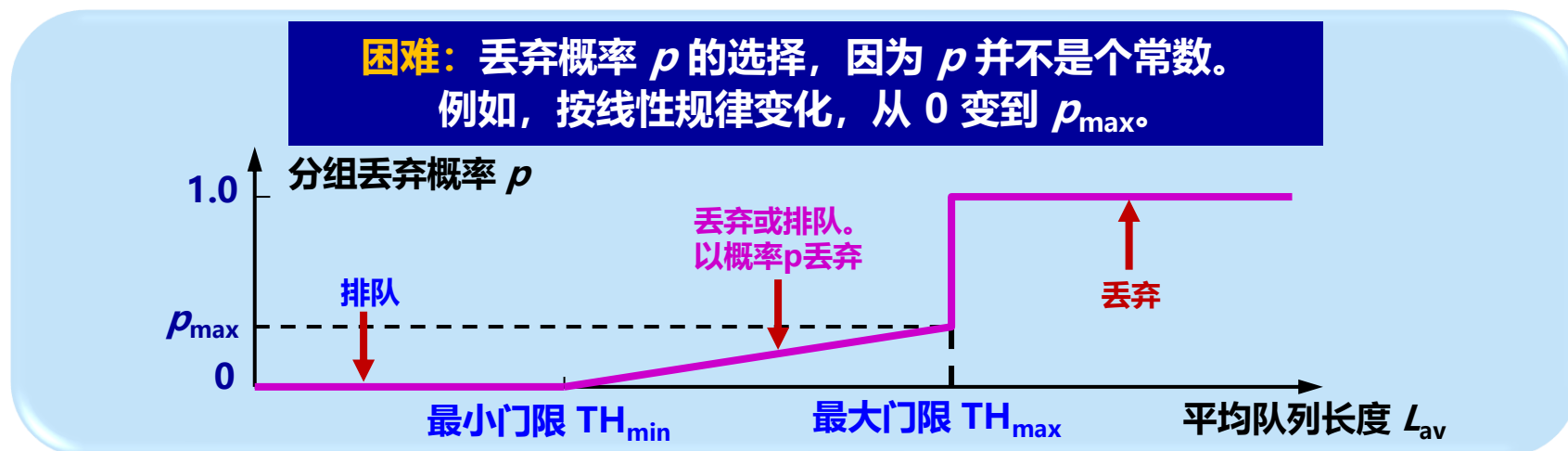
RED 对每一个到达的分组都先计算**平均队列长度 L_{AV}** 。



TCP的拥塞控制

随机早期检测 RED

- 当 $L_{AV} < Th_{min}$ 时, 丢弃概率 $p = 0$
- 当 $L_{AV} > Th_{max}$ 时, 丢弃概率 $p = 1$
- 当 $Th_{min} \leq L_{AV} \leq Th_{max}$ 时, 丢弃概率 p : $0 < p < 1$





TCP的拥塞控制

随机早期检测 RED

- 多年的实践证明，RED 的使用效果并不太理想
- 2015 年公布的 RFC 7567 已经把 RFC 2309 列为“陈旧的”，并且不再推荐使用 RED
- 但对路由器进行主动队列管理 AQM 仍是必要的
- AQM 实际上就是对路由器中的分组排队进行智能管理，而不是简单地把队列的尾部丢弃
- 现在已经有几种不同的算法来代替旧的 RED，但都还在实验阶段



章节大纲

1	运输层协议概述
2	用户数据报协议UDP
3	传输控制协议TCP概述
4	可靠传输的工作原理
5	TCP报文段的首部格式
6	TCP可靠传输的实现
7	TCP 的流量控制
8	TCP的拥塞控制
9	TCP的运输连接管理



TCP的运输连接管理

运输连接的三个阶段

- TCP 是面向连接的协议
- TCP 连接有**三个阶段**:
 - 连接建立
 - 数据传送
 - 连接释放
- TCP 的连接**管理**就是使 TCP 连接的建立和释放都能正常地进行



TCP的运输连接管理

TCP 连接建立过程中要解决的三个问题

- 要使每一方能够确知对方的**存在**
- 要允许双方**协商**一些参数（如最大窗口值、是否使用窗口扩大选项和时间戳选项以及服务质量等）
- 能够对运输实体资源（如缓存大小、连接表中的项目等）进行**分配**

- TCP 连接的建立采用**客户服务器**方式。
- **主动发起**连接建立的应用进程叫做**客户** (client)。
- **被动等待**连接建立的应用进程叫做**服务器** (server)。



TCP的运输连接管理

TCP的连接建立

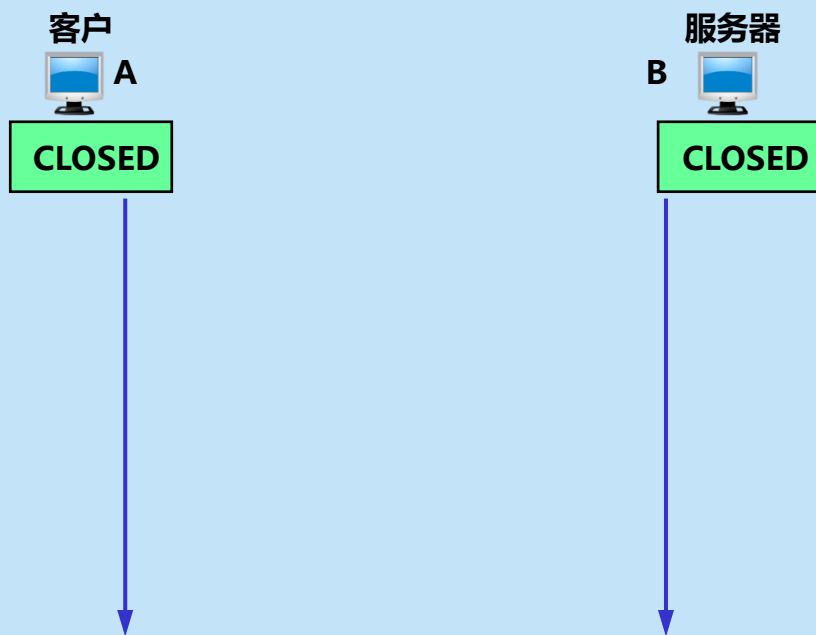
- TCP 建立连接的过程叫做**握手**
- 采用**三报文握手**：在客户和服务端之间交换三个 TCP 报文段，以防止已失效的连接请求报文段突然又传送到了，因而产生 TCP 连接建立错误



TCP的运输连接管理

TCP的连接建立

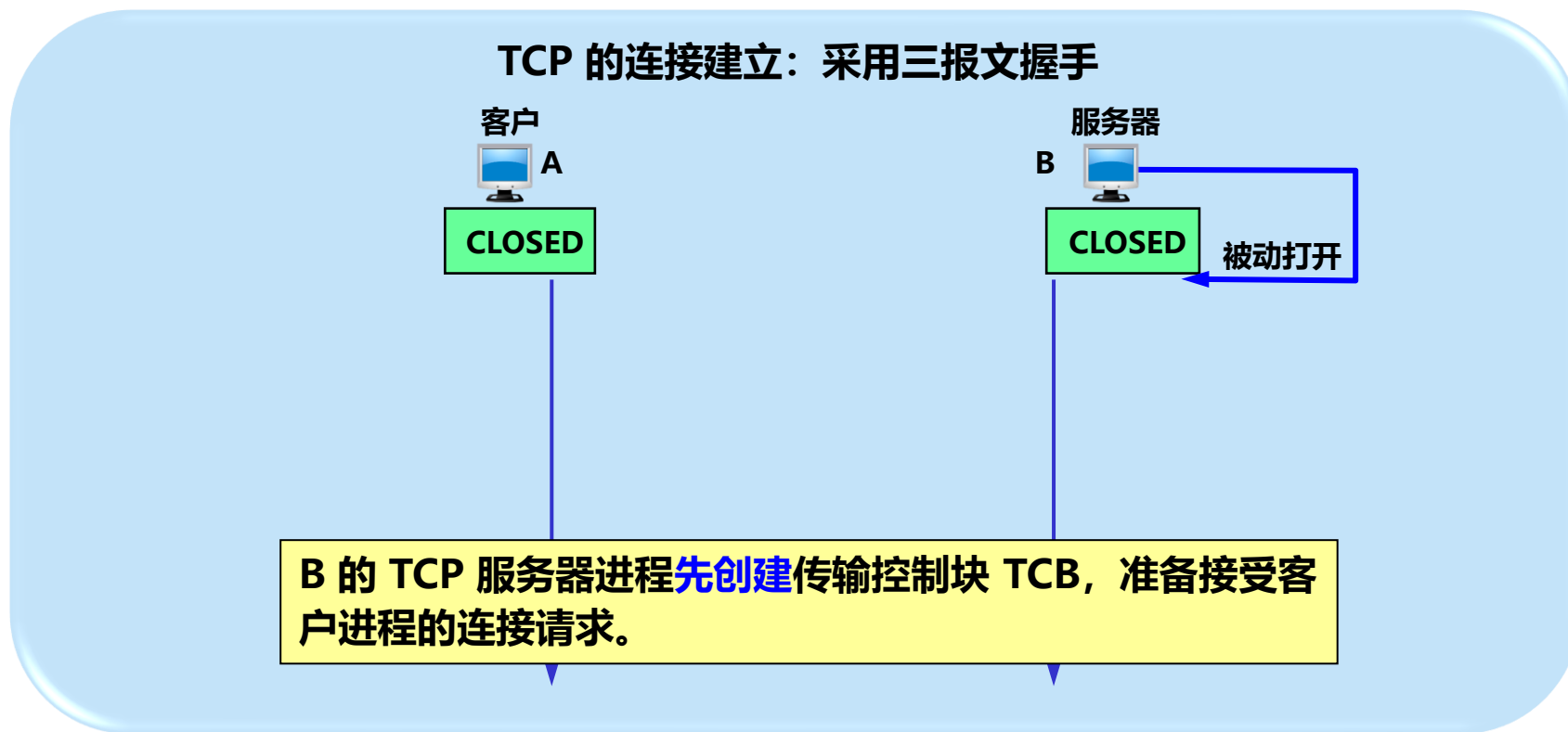
TCP 的连接建立：采用三报文握手





TCP的运输连接管理

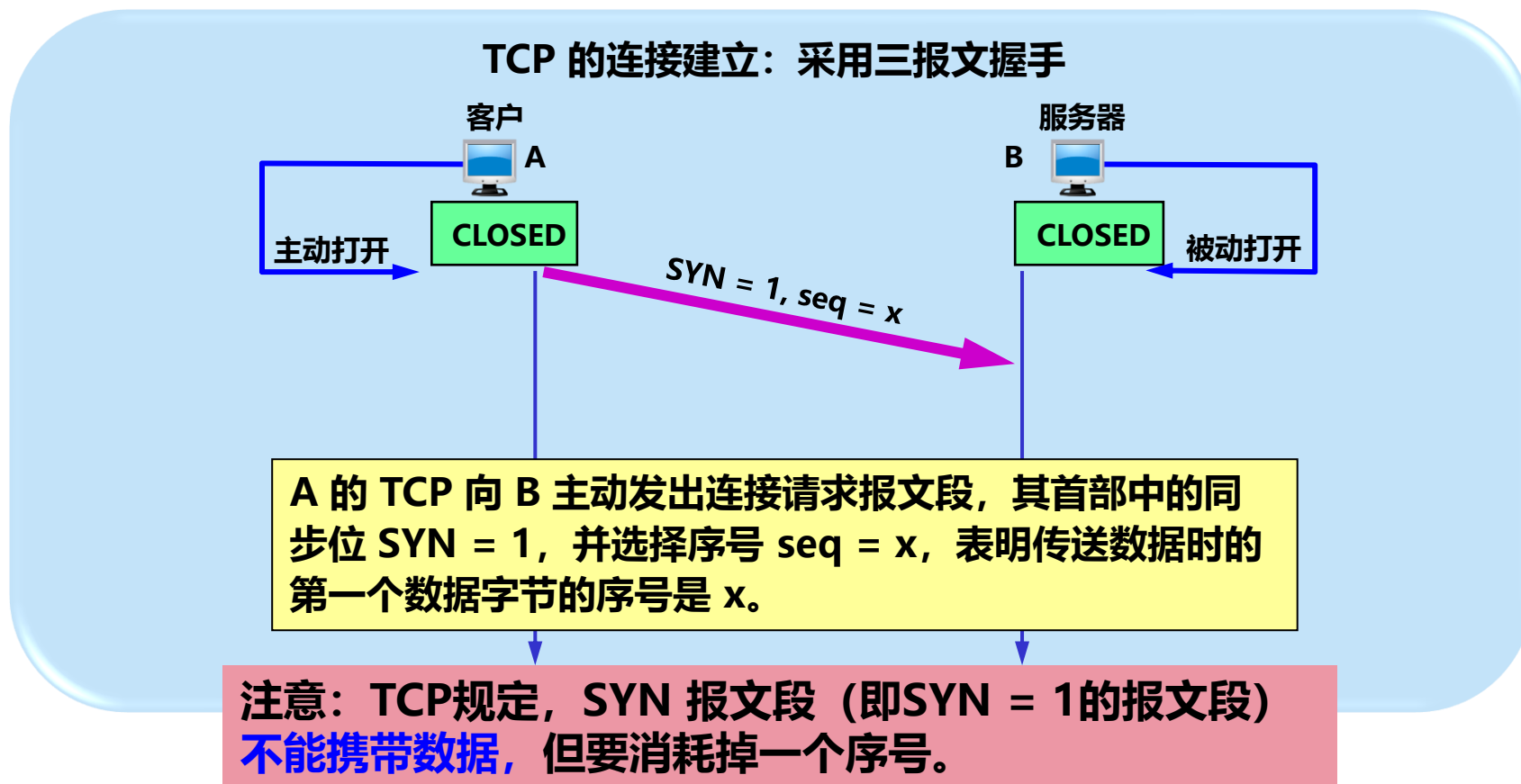
TCP的连接建立





TCP的运输连接管理

TCP的连接建立

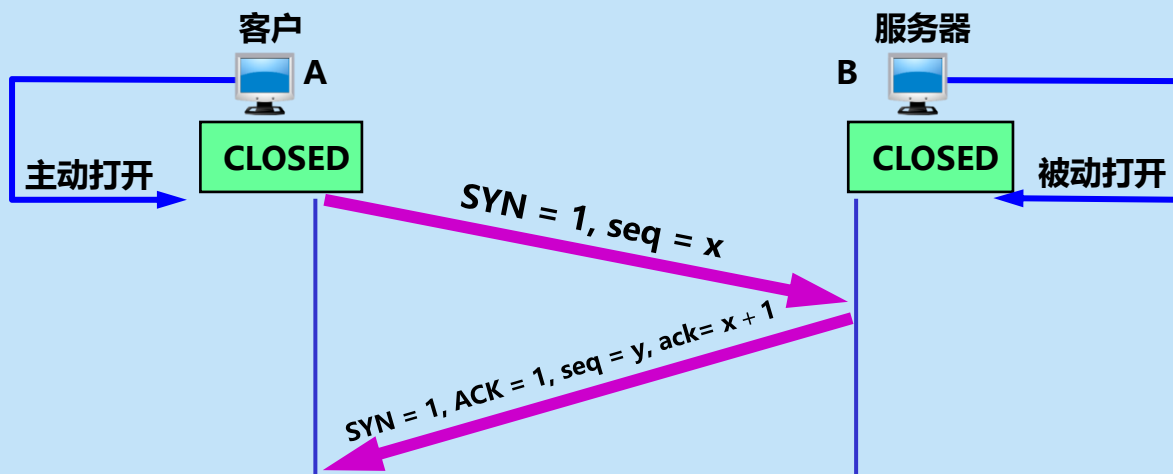




TCP的运输连接管理

TCP的连接建立

TCP 的连接建立：采用三报文握手

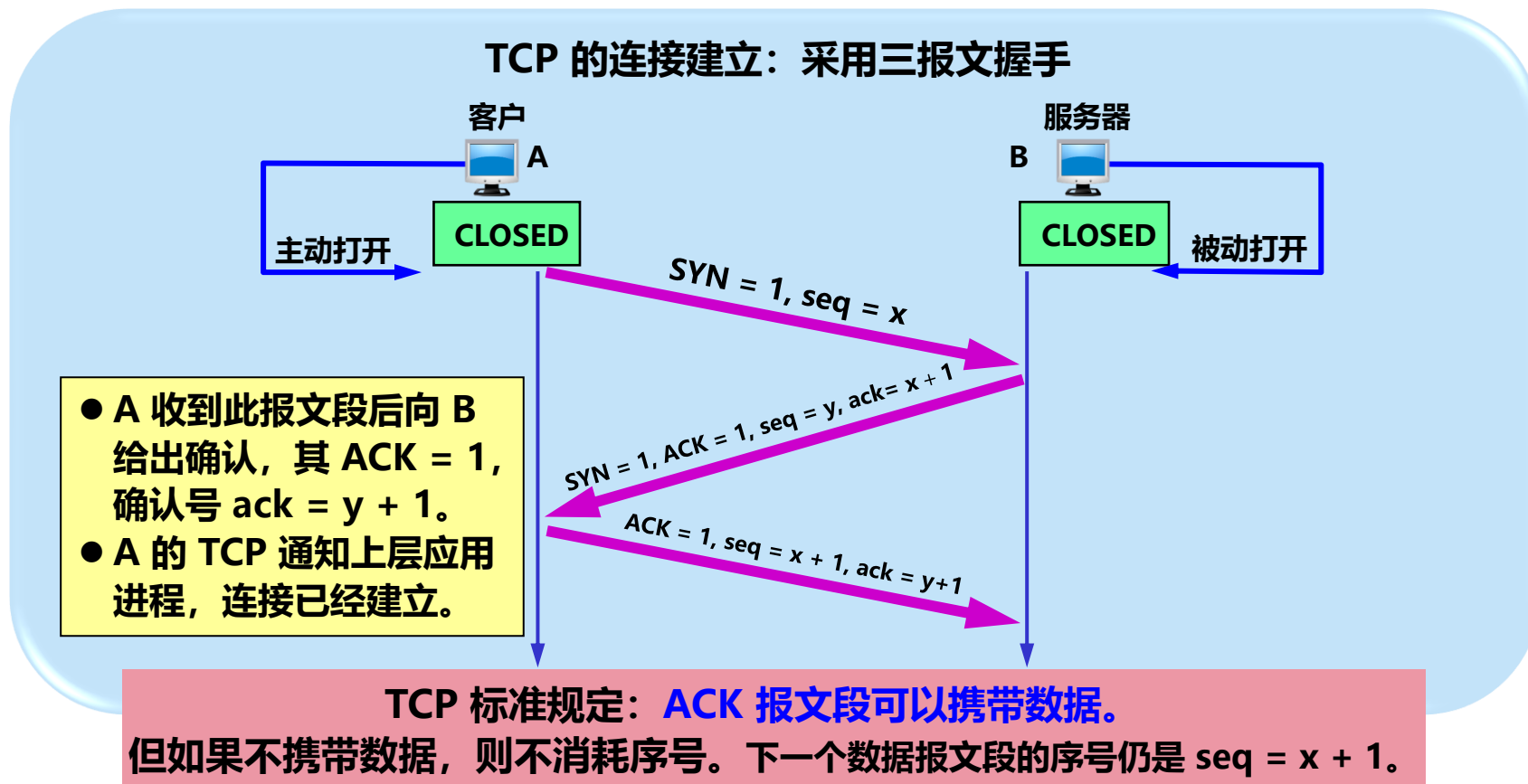


- B 的 TCP 收到连接请求报文段后，如同意，则发回确认。
- B 在确认报文段中应使 $\text{SYN} = 1$ ，使 $\text{ACK} = 1$ ，其确认号 $\text{ack} = x + 1$ ，自己选择的序号 $\text{seq} = y$ 。

这个报文段也不能携带数据，但同样要消耗掉一个序号。

TCP的运输连接管理

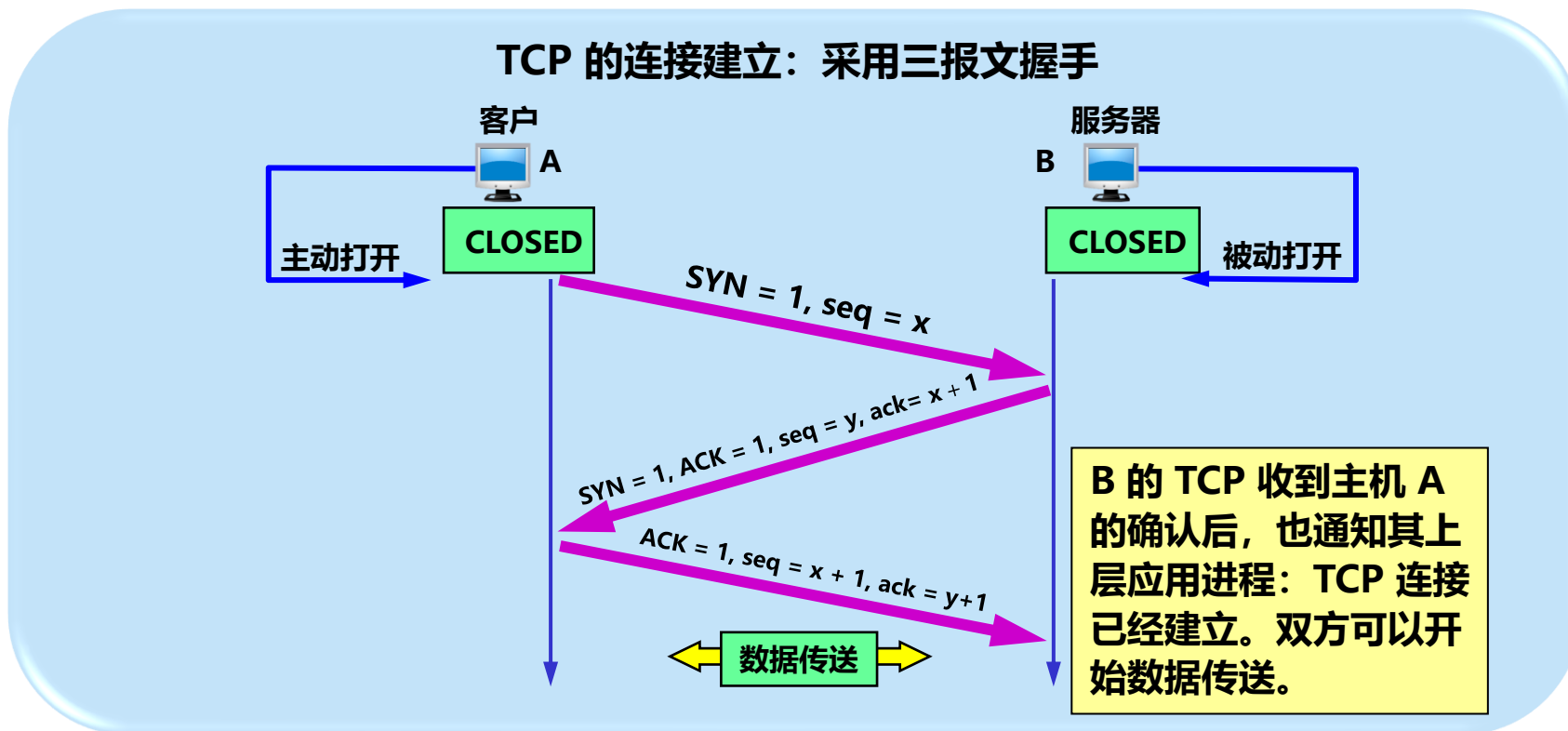
TCP的连接建立





TCP的运输连接管理

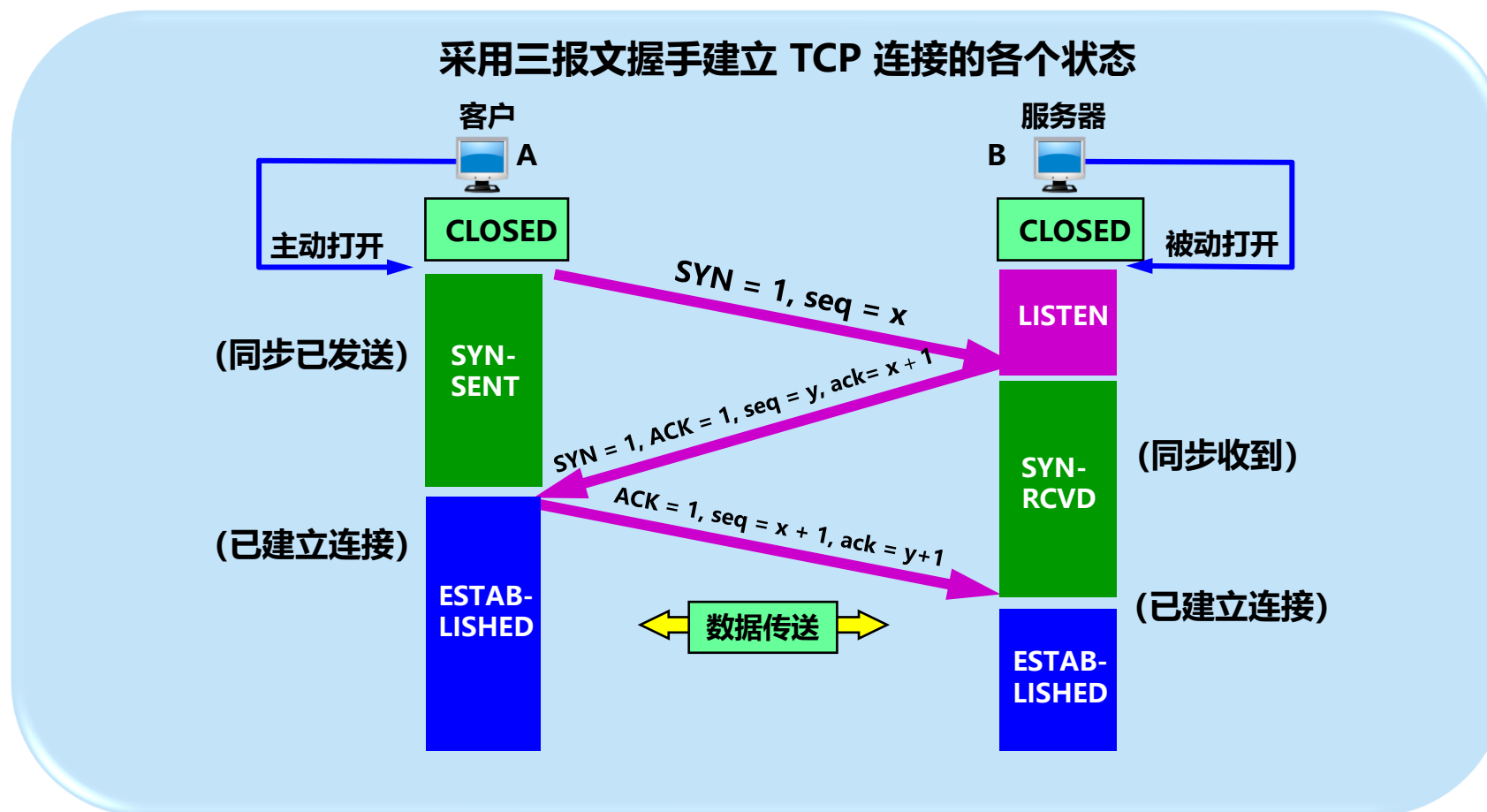
TCP的连接建立





TCP的运输连接管理

TCP的连接建立





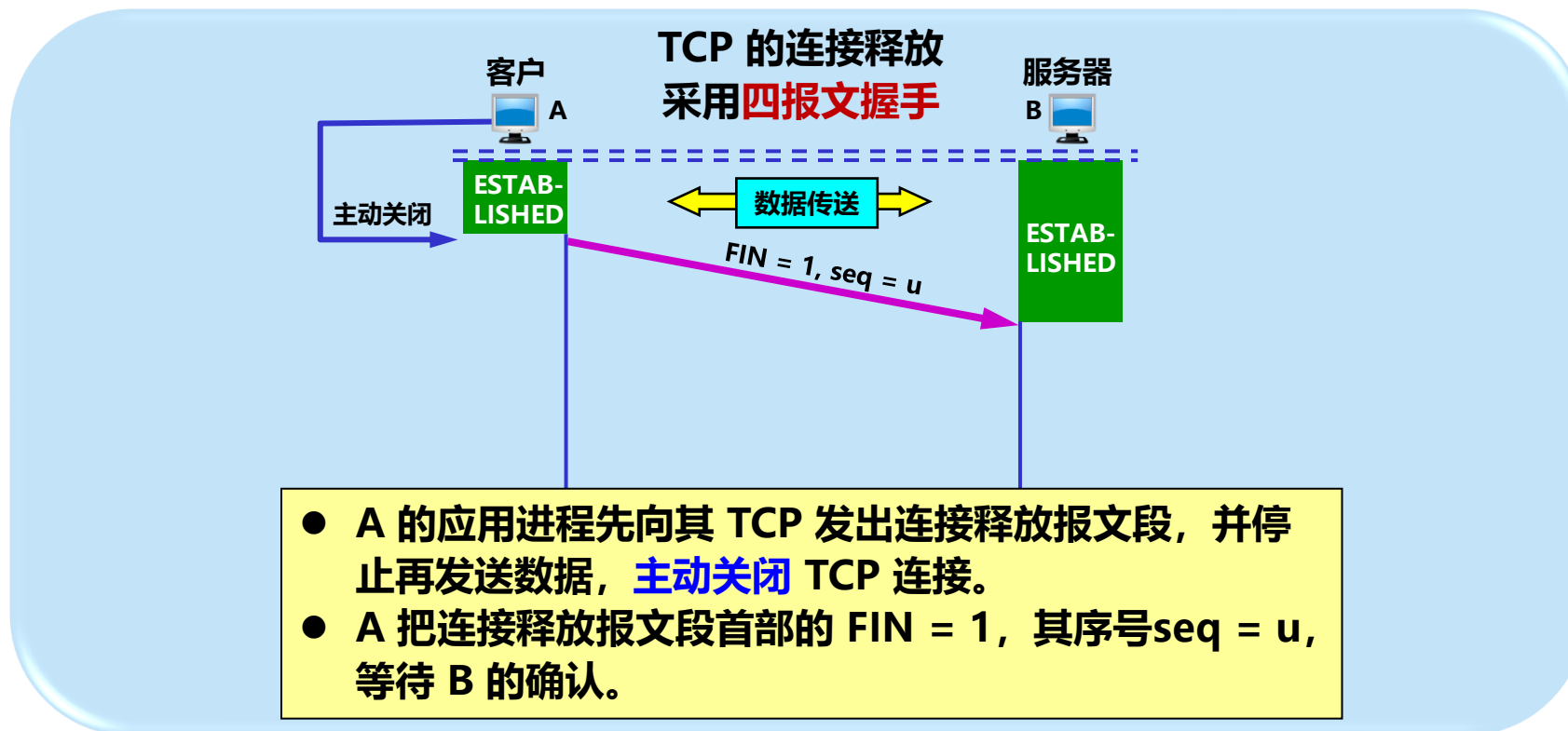
TCP的运输连接管理

TCP连接的释放

- TCP 连接释放过程比较复杂
- 数据传输结束后，通信的**双方**都可释放连接
- TCP 连接释放过程是**四报文握手**

TCP的运输连接管理

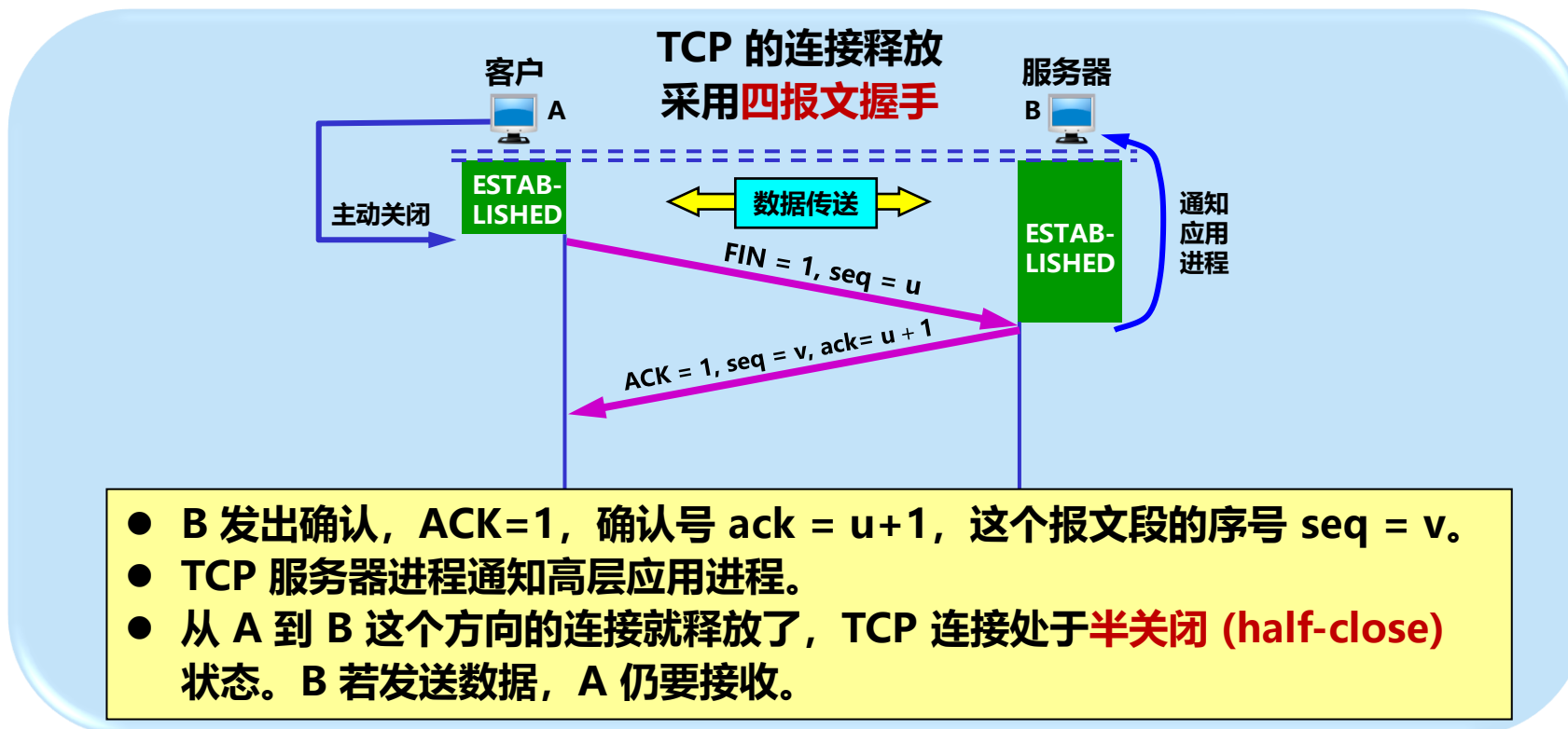
TCP连接的释放



TCP规定：FIN 报文段即使不携带数据，也消耗掉一个序号。

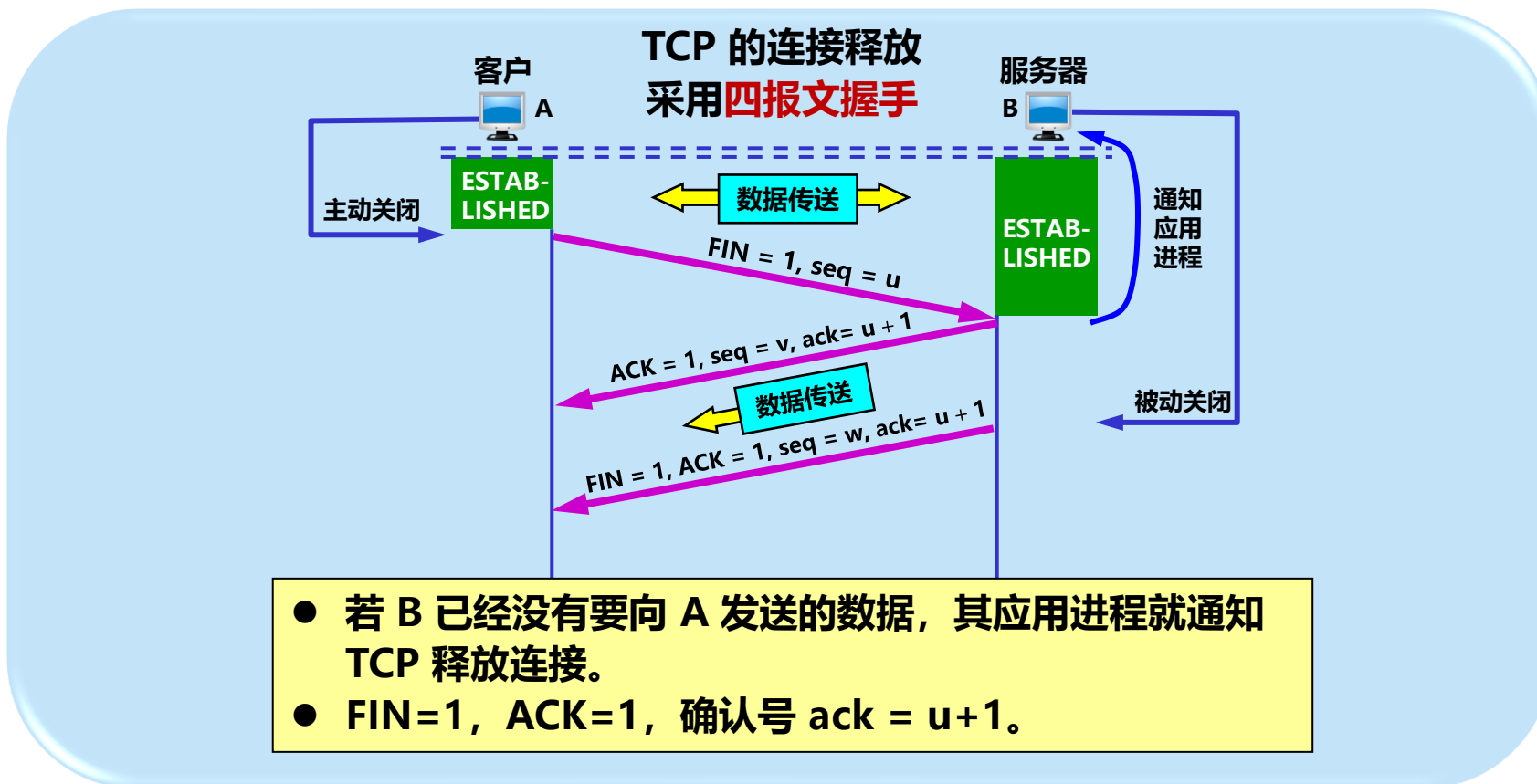
TCP的运输连接管理

TCP连接的释放



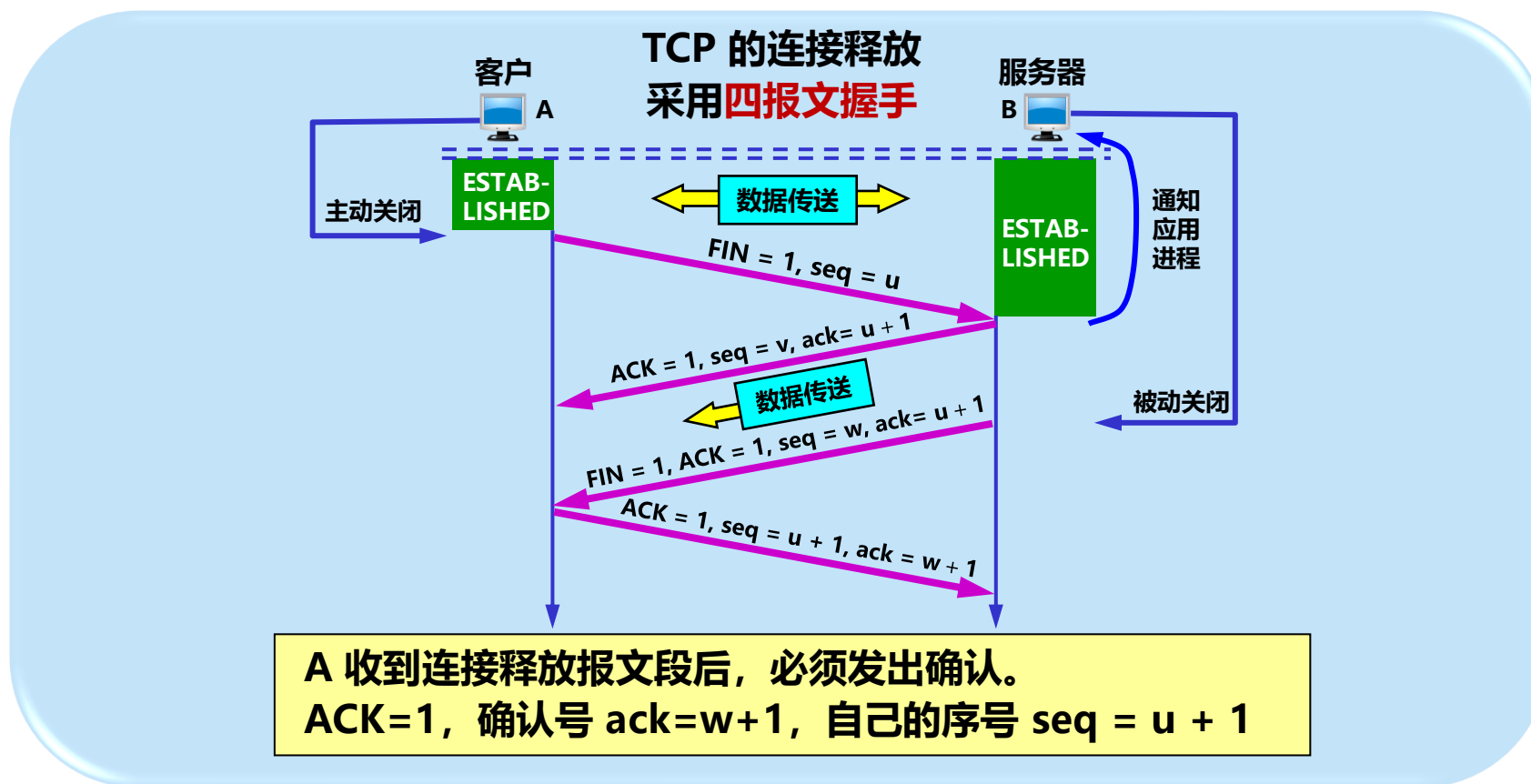
TCP的运输连接管理

TCP连接的释放



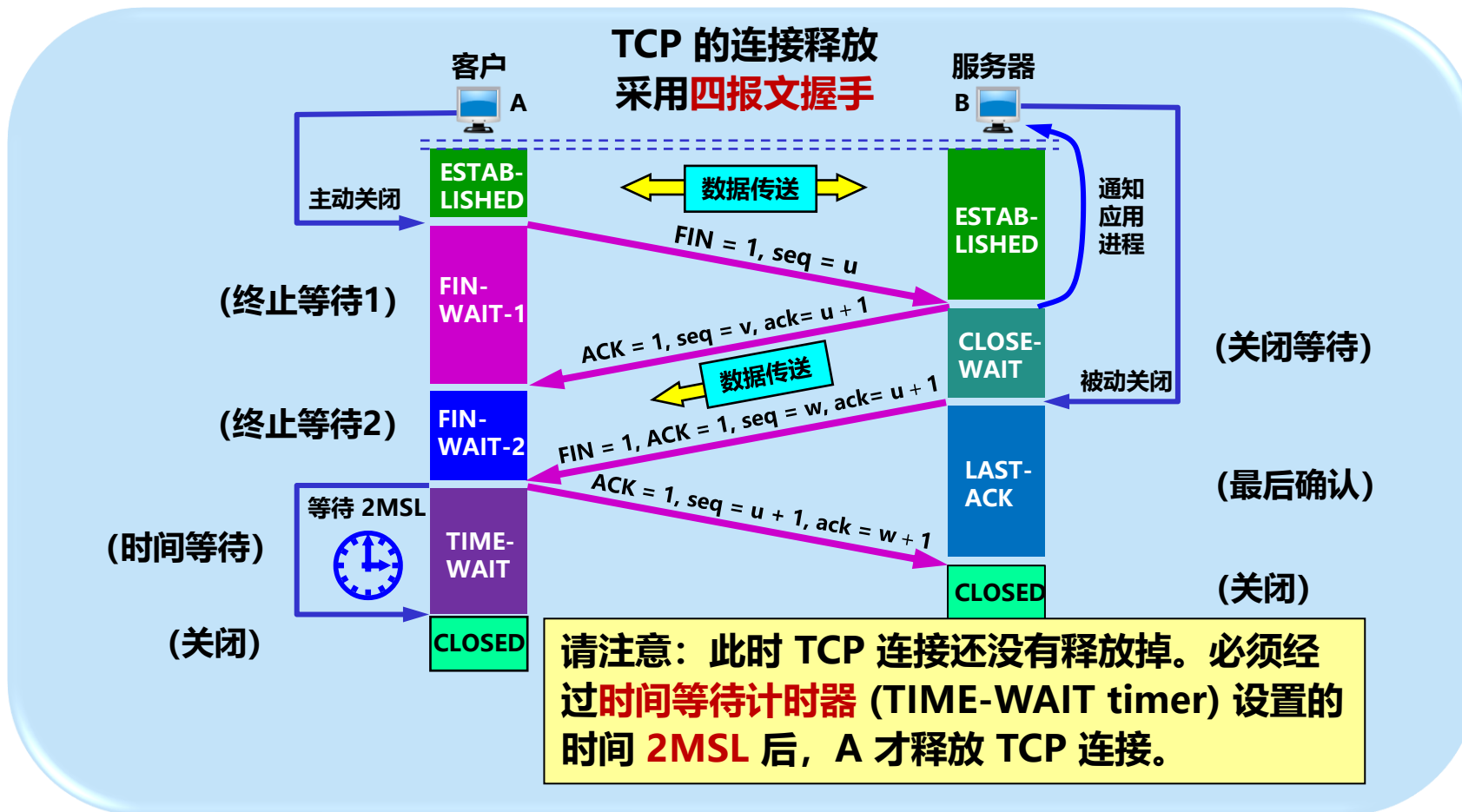
TCP的运输连接管理

TCP连接的释放



TCP的运输连接管理

TCP连接的释放





TCP的运输连接管理

必须等待2MSL的时间

- 第一，保证发送的**最后一个 ACK** 报文段能够到达 B
- 第二，防止“**已失效的连接释放报文段**”出现在本连接中



TCP的运输连接管理

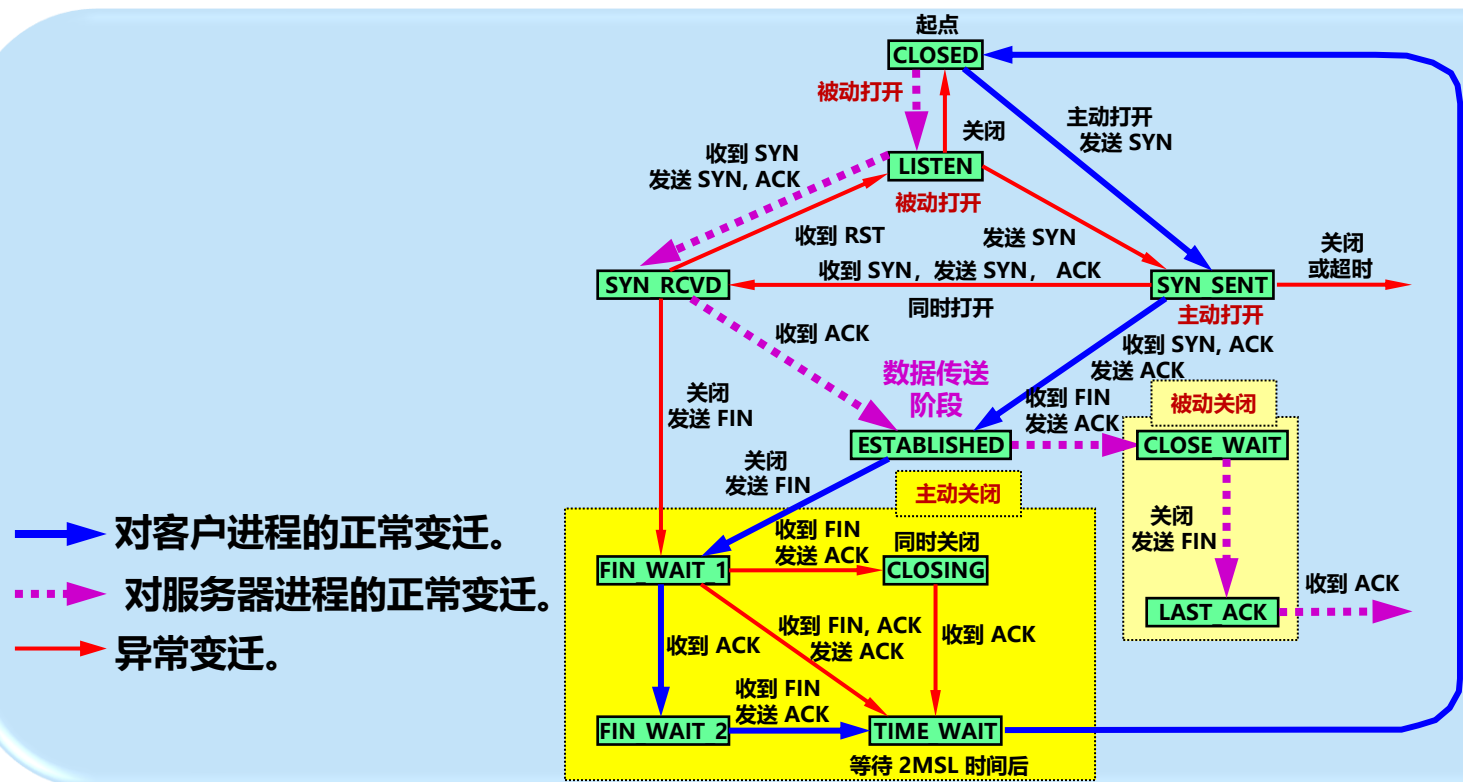
保活计时器

- 用来**防止**在 TCP 连接出现**长时期空闲**
- 通常设置为 2 小时
- 若服务器过了 2 小时还没有收到客户的信息，它就发送**探测**报文段
- 若发送了 10 个探测报文段（每一个相隔 75 秒）还没有响应，就假定客户出了故障，因而就**终止**该连接



TCP的运输连接管理

TCP的有限状态机





TCP的运输连接管理

练习

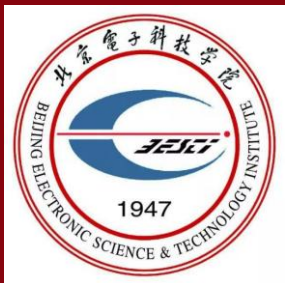
- 主机甲与主机乙建立 TCP 连接时发送的 SYN 段中的序号为 1000，在断开连接时，甲发送给乙的 FIN 段中的序号为 5001，则在无任何重传的情况下，甲向乙已经发送的应用层数据的字节数为？（考研题）
- A. 4002
- B. 4001
- C. 4000
- D. 3999



TCP的运输连接管理

教材作业

- 思考题：5-01—5-12, 5-14, 5-15, 5-23, 5-28, 5-37, 5-38, 5-39, 5-42, 5-66, 5-74
- 计算题：5-49、5-65



谢谢同学们的参与
让我们一起享受计网的饕餮盛宴